The Journal of Supercomputing (2025) 81:911
https://doi.org/10.1007/s11227-025-07415-3

Scheduling DAG-structured workloads based on whale optimization algorithm

Nana Du[1] · Yudong Ji[1] · Chase Wu[2] · Aiqin Hou[1] · Weike Nie[1]

Accepted: 6 May 2025

Abstract
Many computing workloads in big data and machine learning applications are structured as directed acyclic graphs (DAG) and deployed on PC clusters for parallel execution using multiple physical or virtual machines. The scheduling of such workloads is critical to the application performance such as execution time and a plethora of techniques have been developed, taking into account various aspects such as data locality, network bandwidth, and server capability. We formulate DAG-structured workload scheduling as a nonlinear integer programming (NIP) problem and prove it to be NP-complete. Our empirical study reveals a positive correlation between scheduling plan distance (*SPD*) and finish time gap (*FTG*), and based on this finding, we propose a running time gap strategy (RTGS) to tackle this scheduling problem in multiprocessor environments. RTGS follows the main optimization strategy in the family of whale optimization algorithm (WOA). We derive a new function and use a greedy algorithm to generate an effective scheduling plan in RTGS. Extensive experiments with real production traces from Alibaba on simulation environments and realistic Hadoop environments show that our approach significantly improves the stability of WOA when applied to the scheduling problem of DAG-structured workloads, and also reduces the workload completion time by up to 93% in comparison with seven state-of-the-art baseline algorithms.

Keywords Workload scheduling · Directed acyclic graph · Parallel computing · Whale optimization algorithm

1 Introduction

On a PC cluster for production use, a substantial fraction of physical or virtual servers is devoted to long-running workloads (LRWs) [1]. LRWs in standard practice are oftentimes separated into multiple tasks running inside containers, which are generally managed by various resource management middleware programs such as Apache Hadoop YARN [2], Kubernetes/K8s [3], and Apache Mesos [4].

Extended author information available on the last page of the article

Published online: 28 May 2025

LRWs typically contain tasks that may run in parallel, or must respect execution dependency as they need the data generated by some other tasks. In many big data and machine learning applications such as Spark-based data processing or large language model training, a job of LRWs is oftentimes modeled as a directed acyclic graph (DAG) [5, 6], in which nodes represent tasks, and edges represent data transfers between tasks. The scheduling of such DAG-structured workloads is critical to the application performance such as execution time and a plethora of techniques have been developed in the literature, taking into account various aspects such as data locality, network bandwidth, and server capability. Regardless of the tremendous efforts made in industry and the research community, scheduling DAG-structured workloads still remains as a challenging problem. A too scattered scheduling plan always leads to excessive data transfer time from preceding tasks, and a too centralized scheduling plan may cause long task waiting time.

LRW scheduling poses some unique challenges, particularly when leveraging machine learning (ML) techniques. ML-based approaches, such as reinforcement learning or neural network-based schedulers, aim to learn optimal scheduling policies by modeling complex task dependencies and resource dynamics [6]. However, they face significant hurdles: (1) the need for extensive training data, which may not always be available for diverse workload patterns; (2) high computational overhead during training and inference, which can be impractical for real-time scheduling in production environments; and (3) limited generalizability across heterogeneous clusters or varying DAG structures. In contrast, heuristic-based methods, such as genetic algorithms (GA), ant colony optimization (ACO), or whale optimization algorithm (WOA), rely on predefined rules or bioinspired strategies to explore the solution space. These methods are generally computationally lighter and easier to implement but often suffer from suboptimal convergence or sensitivity to parameter settings, leading to inconsistent performance across different workload scenarios. For instance, WOA excels in balancing exploration and exploitation for continuous optimization problems but struggles to handle the discrete, high-dimensional nature of LRW scheduling, where direct distance metrics between scheduling plans do not accurately reflect their true relationship.

In this work, we formulate DAG-structured workload scheduling as a nonlinear programming (NLP) problem, referred to as long-running workload task scheduling (LRW-TS), and prove it to be NP-complete. Our empirical study reveals a positive correlation between scheduling plan distance (*SPD*) and finish time gap (*FTG*), and based on this finding, we propose a running time gap strategy (RTGS) to tackle this scheduling problem in parallel computing environments. RTGS follows the main optimization strategy in the family of whale optimization algorithm (WOA) [7]. WOA is a swarm intelligence algorithm inspired by the humpback hunting method in 2016 and has been successfully applied to solve practical optimization problems [8]. WOA performs better than other bioinspired optimization algorithms in terms of convergence speed and balance between exploration and exploitation [9]. In WOA, a humpback hunts according to the physical distance. However, in this work, a humpback is a vector with N dimensions, so it is not straightforward to measure the distance between two humpbacks, or the distance between two different scheduling plans. Unlike ML-based approaches that require

extensive training to model such distances, or traditional heuristic methods that apply simplistic distance metrics, RTGS leverages the *FTG–SPD* correlation to redefine the concept of distance in a computationally efficient manner, combining the strengths of heuristic optimization with domain-specific insights. To integrate WOA into RTGS, we are facing two key challenges:

1. How to measure the distance between different scheduling plans with N dimensions?
2. How to generate a new scheduling plan with a specific distance to another scheduling plan?

To address the first challenge, we propose to use *FTG*, which is a straightforward metric, to measure the distance between different scheduling plans. However, this metric does not allow us to directly generate new scheduling plans. To overcome this issue, we further convert *FTG* to *SPD* according to the revealed positive correlation between them, thus addressing the second challenge. We design a greedy algorithm to create a new scheduling plan and control its distance to the old one. The greedy algorithm is based on a series of derivation functions as well as the correlation between *FTG* and *SPD*. This approach contrasts with ML-based methods, which may fail to produce precise scheduling plans due to generalization issues, and heuristic methods such as GA and ACO, which often lack fine-grained control over plan generation. By integrating a domain-specific distance metric with a greedy strategy, RTGS achieves superior stability and performance without the computational burden of model training in ML.

Extensive experiments with real production traces from Alibaba on simulation environments and realistic Hadoop environments show that our approach significantly improves the stability of WOA when applied to the LRW-TS problem. Furthermore, it reduces the workload completion time up to 93% in comparison with seven state-of-the-art baseline algorithms. It is worth pointing out that for workloads without a DAG structure, all of these algorithms perform relatively well, and RTGS still yields the best performance due to its strong search ability.

In summary, we make the following key contributions in this work:

1. We formulate LRW-TS as an NLP problem and provide a rigorous proof of its NP-completeness.
2. We reveal that there exists a positive relationship between *SPD* and *FTG*, which guides the design of our approach.
3. We perform mathematical reasoning and transform the uncontrollable iteration process into a strongly controllable procedure.
4. We propose RTGS based on WOA to tackle LRW-TS and experimental results using real-life workload traces show that RTGS reduces execution time by up to 93% in comparison with state-of-the-art baseline schedulers.
5. The introduction of *SPD* and the design principle of RTGS open a new direction to tackle task scheduling problems.

The rest of this paper is structured as follows. Section 2 conducts a survey of related work and points out a critical issue when swarm intelligence is used for task scheduling. Section 3 constructs the cost models and formulates the LRW-TS problem using a nonlinear programming model. The computational complexity of this problem is also analyzed. Section 4 reveals the positive correlation between *FTG* and *SPD*, and details the design of the RTGS algorithm. Section 5 evaluates the performance of our approach compared with state-of-the-art algorithms. We conclude our work and sketch a plan of future work in Sect. 6.

2 Related work

There exist numerous efforts in the literature for workload scheduling, which can be roughly divided into two categories based on the techniques they use: machine learning and heuristic method. We conduct a survey of cutting-edge techniques that are directly related to our work.

Machine learning In recent years, ML techniques have been increasingly used to perform heuristic search. Symphony framework [10] is a domain-driven Bayesian reinforcement learning method to model resource dependencies from system architectures. In order to generate highly efficient policies automatically regardless of different workload characteristics, Decima [6] employed reinforcement learning and neural networks to design scheduling algorithms without human intervention. Simultaneous spike in resource usage from two workloads running on the same machine can create performance degradation, and idle resources in a machine lead to waste. To solve this problem, Mondal et al. [11] proposed a deep reinforcement learning-based method to simulate resource usage patterns of time-varying workloads. Based on the simulation results, they developed a technique for production workload scheduling. Wi-SeDB [12] is a learning-based framework for holistic workload scheduling, which is customized to workload characteristics. Chen et al. [13] proposed Atos scheduling framework for multi-node-GPU systems, which provides PGAS-style memory operations within and between nodes.

Heuristic method There are several scheduling heuristics commonly employed in various systems. These techniques include first-in-first-out (FIFO) [14], capacity scheduler [15], fair scheduler [16], and round robin scheduler [17], which, as fundamental building blocks [18, 19], are often customized or combined to meet specific requirements and optimize resource allocation.

The liger algorithm [20] employs a scheduling algorithm with hybrid synchronization and resource contention anticipation to optimize computation and communication kernel scheduling in multi-GPU architectures. By facilitating pipelined and parallel processing of multiple graph snapshots on GPUs, PiPAD [21] optimizes dynamic graph neural networks (DGNN) training, addressing memory and data-related challenges in task scheduling.

A programming model introduced in [22] decouples load balancing from work processing, allowing both static and dynamic schedules. Stec et al. [23] assumed normal

distribution in job processing time and proposed a branch-and-price approach to maximize the probability of jobs finishing before a common deadline.

Contract algorithms have strict query deadlines and lack interruptibility. To adapt them for interruptible systems, the work in [24] explores achieving desired performance objectives within specific time frames. For cloud platforms, pre-collected job scheduling is formulated as a prediction optimization problem and solved by the CUC algorithm in [25]. The multi-objective Harris hawks optimization-based task scheduling algorithm [26] aimed at optimizing task scheduling in cloud–fog computing environments to reduce delay and energy consumption. Graph-oriented simulated annealing (GOSA) algorithm [27] is to address task graph scheduling in dynamically reconfigurable hardware systems, effectively managing task dependencies and reconfiguration overhead.

OpenCilk [28] supports rapid prototyping for exploring design choices in language abstraction, compilation strategy, and runtime scheduling, using a work-stealing runtime scheduler for effective task distribution. Godet et al. [29] scheduled jobs on machines with unit resource constraints from satellite data processing.

Multiple resource allocation problems are addressed in [30–33]. Long-running applications were scheduled using an optimization-based approach in [34]. A scalable scheduling framework was deployed on Microsoft Azure [35]. The Firmament scheduler [36] scales a centralized data center to over ten thousand machines with subsecond placement latency. The smarter function scheduler (SFS) [37] enhances performance by trading off the duration of long applications using user-space policies.

Swarm intelligence has emerged as a popular approach to solving task scheduling problems in various fields [38–40]. However, one critical issue with these algorithms is that every entity or scheduling plan needs to calculate and determine its next position during each iteration. In this process, each dimension of a scheduling plan is involved, but the value of each dimension only represents the number of servers and does not provide much valuable information. To address this issue, we reveal and incorporate a positive relationship between *FTG* and *SPD* into distance calculation, and generate new scheduling plans that have desired distances from the previous ones. This approach avoids relying solely on the number of servers and allows us to consider other important factors in task scheduling such as task completion time and data transfer speed. We further design a greedy algorithm to improve the proposed approach by incorporating additional constraints that are specific to the task scheduling problem. These improvements result in better performance and efficiency in solving complex task scheduling problems.

3 Problem formulation

3.1 Cost models

A job of LRWs is generally modeled as a directed acyclic graph $G_T\{T, E_T, W, M, D\}$, where $T = \{t_1, t_2, \ldots, t_n\}$ denotes a set of n tasks, $E_T = \{(t_i, t_j) | t_i, t_j \in T\}$ denotes a set of directed edges that specify the data

transmission (DT) direction, $W = \{w_1, w_2, \ldots, w_n\}$ denotes a set of running time costs for all tasks, $M = \{m_1, m_2, \ldots, m_n\}$ is a set of resources required for all tasks, and $D = \{d_{ij} | t_i, t_j \in T\}$ is a set of edge weights that specify the data size transferred from a preceding task t_i to its succeeding task t_j.

We model a parallel computing platform as an undirected graph $G_H\{H, E_H, C, B\}$, where $H = \{h_1, h_2, \ldots, h_m\}$ denotes a set of m physical or virtual servers, $E_H = \{(h_i, h_j) | h_i, h_j \in H\}$ denotes a set of undirected edges, $C = \{c_1, c_2, \ldots, c_m\}$ denotes a set of available resources on all servers in H, and $B = \{b_{ij} | h_i, h_j \in H\}$ denotes a set of bandwidths for all edges in E_H. In this paper, c_i represents the available CPU on server h_i, and can be expanded to include other resources.

The time cost of scheduling a job of LRWs is influenced by multiple factors, which collectively determine the job's finish time. These factors include:

- *Task execution time* For each task $t_i \in T$, the execution time $w_i \in W$ depends on the task's time complexity and the processing capability of the assigned server $h_j \in H$. The execution time is constrained by the available CPU resources c_j, and insufficient resources may lead to task queuing or extended execution.
- *Data transmission time* For each edge $(t_i, t_j) \in E_T$, the transmission time is determined by the data size $d_{ij} \in D$ and the available bandwidth $b_{kl} \in B$ between two servers h_k and h_l that execute t_i and t_j, respectively. If tasks are assigned to different servers, the transmission time may vary due to different network latency and bandwidth availability.
- *Waiting time* Tasks with dependencies must wait for their preceding tasks to complete and for data to be transferred. Waiting time is exacerbated in scenarios with resource contention, where servers are overloaded, or when tasks are unevenly distributed, leading to idle periods on some servers while others are kept busy.

The job's finish time, defined as the final completion time of all tasks, is a critical performance metric. However, optimizing this metric involves balancing scheduling cost with load balancing. Scheduling cost, primarily driven by data transmission times, could be minimized by co-locating dependent tasks on the same server to reduce data transfers. However, such colocation can lead to resource contention and uneven load distribution, hence increasing waiting and execution times due to overloaded servers. Conversely, distributing tasks across servers to achieve load balancing may increase data transmission times, as data must traverse network links with limited bandwidth. We aim to find an optimal scheduling plan that minimizes the job's finish time by carefully balancing these conflicting factors, taking into account task dependencies, server capacities, and network constraints.

For illustration, we plot in Fig. 1 a job of five tasks and a simple cloud-based computing platform of three virtual servers. In the model of DAG-structured tasks, nodes $\{t_1, t_2, t_4\}$ without any preceding tasks are entry nodes, and node $\{t_5\}$ without any succeeding task is an exit node. The directed edges such as (t_1, t_3) in DAG-structured tasks indicate the data transmission direction. In the model of parallel

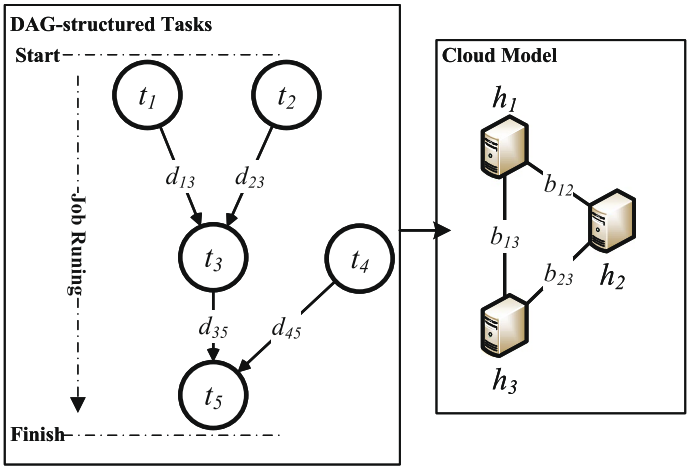


Fig. 1 Models of tasks with a DAG structure executed in clouds

Table 1 Two example scheduling plans

Tasks	t_1	t_2	t_3	t_4	t_5
Plan one	h_1	h_1	h_1	h_1	h_1
Plan two	h_1	h_3	h_2	h_2	h_2

computing, each node has limited resources c_i of CPU and the link between servers h_i and h_j has bandwidth b_{ij}.

This paper explores an effective scheduling plan of tasks in a long-running job to minimize job's finish time. Table 1 tabulates two scheduling plans for the job in Fig. 1. For simplicity, we suppose that the time for task execution and data transmission time is 1 and the start time to execute the job is 0. In a scenario where servers can execute all tasks concurrently, as shown in Fig. 2a, plan one has a finish time of 3, and plan two has a finish time of 4. That is because $\{t_1, t_2, t_4\}$ finish within one unit of interval and there is no DT time in plan one, but in plan two, $\{t_1, t_4, t_2\}$ are mapped to $\{h_1, h_2, h_3\}$, respectively. After one unit of interval, data needs to be transferred from h_1 and h_3 to h_2. This example illustrates the trade-off between scheduling cost and load balancing: plan one minimizes data transmission time by co-locating tasks on h_1, but risks resource contention if h_1's capacity c_1 is insufficient, while plan two distributes tasks to balance workloads, but incurs additional data transmission time due to data transfer across servers. However, in the other scenario shown in Fig. 2b where one server can only execute one task at a time, plan one has a finish time of 5 and the finish time of plan two is still 4. Here, plan one suffers from an increased waiting time due to sequential execution on h_1, whereas plan two benefits from parallel execution across servers, illustrating how load balancing reduces the job's finish time in resource-constrained environments. These simplistic examples indicate that different scheduling plans are preferred in different computing environments with sufficient or insufficient resources.

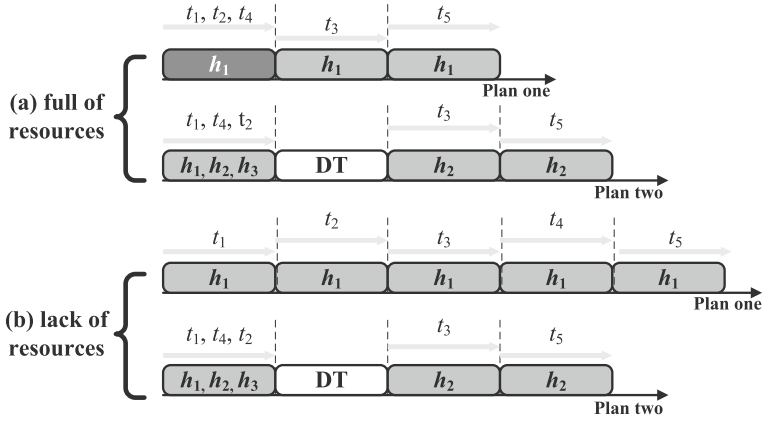


Fig. 2 Job finish time in various scenarios with different scheduling plans

3.2 Problem definition

In this section, we analyze the execution dynamics of DAG-structured workloads, formulate a workload scheduling problem, LRW-TS, as a nonlinear programming problem, and prove it to be NP-complete.

For task t, if it is an entry node, the earliest start time $EST(t)$ is 0; otherwise, $EST(t)$ is the maximum arriving time (AT) of the data generated from its preceding tasks:

$$EST(t) = \begin{cases} 0, & \text{if } t = entry\ node; \\ \max_{t' \in precede(t)} AT(t', t), & \text{else.} \end{cases} \tag{1}$$

The arriving time of the data generated from preceding task t_i is the actual finish time (AFT) of the preceding task t_i plus data transmission time ($Trans(t_i, t_j)$) from the machine running preceding task t_i to the machine running succeeding task t_j:

$$AT(t_i, t_j) = AFT(t_i) + Trans(t_i, t_j). \tag{2}$$

If preceding task t_i is executed on the same machine as succeeding task t_j, the data transmission time is 0; otherwise, the data transmission time from the machine running preceding task t_i to the machine running succeeding task t_j is calculated as

$$Trans(t_i, t_j) = \begin{cases} 0, & \text{if } host(t_i) = host(t_j) \\ \dfrac{d(t_i, t_j)}{b(host(t_i), host(t_j))}, & \text{else} \end{cases} \tag{3}$$

where $d(t_i, t_j)$ denotes the data size sent from preceding t_i to succeeding t_j, $b(host(t_i), host(t_j))$ denotes the bandwidth from the machine running preceding

task t_i to the machine running succeeding task t_j, and $host(t_i)$ denotes the server running task t_i.

If there are sufficient available resources ($AvailC$) on server s processing task t at the time of $EST(t)$, the actual start time $AST(t)$ of task t is the earliest start time of task t; otherwise, task t has to wait for sufficient resources ($SufR$) on server s to execute task t. We compute $AST(t)$ as

$$AST(t) = \begin{cases} EST(t), & \text{if } AvailC(h, EST(t)) > m_t \\ SufC(h,t), & \text{else} \end{cases} \tag{4}$$

where $AvailC(h, EST(t))$ denotes the available CPU on server h at the time of $EST(t)$. We calculate $AvailC$ on server h at a specific time as

$$AvailC(h, time) = AllCpu(h) - UsedC(h, time), \tag{5}$$

where $UsedC(h, time)$ denotes the occupied CPU on server h at $time$ and is calculated as

$$UsedC(h, time) = \sum_{t \in T} m_t \cdot x(t,h) \cdot \max\left\{0, \frac{AFT(t) - time}{|AFT(t) - time|}\right\}, \tag{6}$$

where m_t is the CPU required for task t, $x(t,h)$ is 1 if task t is scheduled on server h, otherwise is 0, and $SufC(h,t)$ denotes the time when there are sufficient resources to process task t on server h, calculated as

$$SufC(h,t) = \min\left\{ AFT(t') \middle| \begin{array}{l} t' \in T, \\ x(t',h) = 1, \\ AvailC(h, AFT(t')) \geqslant m_t \end{array} \right\}. \tag{7}$$

There are several other constraints on workload execution as follows:

$$host(t) = \sum_{h \in H} x(t,h) \cdot h, \quad t \in T \tag{8}$$

$$AFT(t) = AST(t) + w_t, \quad t \in T \tag{9}$$

$$x(t,h) = 0 \text{ or } 1, \quad t \in T, h \in H \tag{10}$$

$$\sum_{h \in H} x(t,h) = 1, \quad t \in T, \tag{11}$$

where $host(t)$ is the processor number to execute task t, $AFT(t)$ is the actual finish time of task t and is equal to the actual start time plus the running time of task t. When task t is assigned to server h, $x(t,h) = 1$; otherwise, $x(t,h) = 0$. Equation (11) ensures that any task is assigned to only one server.

The objective of the scheduling algorithm is to minimize the final finish time (FFT) of a long-running job,

$$FFT = \max \{AFT(t) |\ t \in T\}. \tag{12}$$

For convenience, we tabulate the parameters used in the problem formulation in Table 2. We formally define the long-running workloads task scheduling (LRW-TS) problem as follows:

Definition 1 Given a DAG-structured job $G_T\{T, E_T, W, M, D\}$ and a parallel computing platform $G_H\{H, E_H, C, B\}$, we wish to compute a scheduling plan that minimizes *FFT* while respecting the execution dependency and satisfying the resource constraints as specified from Eqs. (1) to (11).

3.3 Model validation using an optimizer

To verify the correctness, optimality, and scientific validity of the models constructed above, we conduct a series of experiments using a commercial optimization solver, IBM ILOG CPLEX [41]. For reproducibility, we provide the following supplementary materials in a GitHub repository:[1] The CPLEX model file (.lp format) encoding the mathematical formulation; the data files used for the synthetic DAG test instances; the output logs and solutions returned by the CPLEX solver; and a script to parse and visualize the scheduling plan.

We translate the mathematical formulation, including all decision variables, constraints (Eqs. 1–11), and the objective function (Eq. 12), into an Integer Programming model compatible with CPLEX. Each task-to-server mapping variable $x(t, h)$ is treated as a binary decision variable, and all time-related equations, such as AST(t), AFT(t), and Trans(t_i, t_j), are formulated as constraints and expressions within the optimization model. The objective function is to minimize the final finish time FFT of the entire DAG workload.

To make the validation tractable within the computational limitations of exact solvers, we use a small-scale synthetic DAG consisting of 10 tasks and three servers. Task execution times are randomly assigned as integer values between one and five time units. Resource demands for each task are set between one and four CPU units. Inter-task transfer volumes (in data units) are chosen from the range of 10 to 100. Server capacities range from four to eight CPU units, and bandwidths between server pairs are set between 50 and 100 Mbps. We run the CPLEX solver with default settings and a time limit of 600 s per instance. The solver successfully returns optimal solutions for all test cases within seconds, indicating that the models are well constructed and the problem is computationally solvable. The experimental results using CPLEX indicate that the constructed mathematical models in LRW-TS are valid and capable of producing globally optimal scheduling plans. This confirms the theoretical rigor of our problem formulation and reinforces the credibility of the performance evaluations of our proposed scheduling algorithms.

[1] https://github.com/AIprinter/task-scheduling.

Table 2 Parameters in the problem formulation

Parameters	Definition
G_T	A job with a directed acyclic graph of tasks
n	The number of tasks
T	A set of all tasks
t_i	The i-th task
E_T	A set of dependency edges in G_T
(t_i, t_j)	The dependency edge from task t_i to task t_j
W	A set of running times for all tasks
w_i	The running time of t_i
M	A set of resources needed for all tasks
m_i	Resources needed for t_i
D	A set of transferred data sizes along dependency edges in E_T
d_{ij}	The transferred data size of edge (t_i, t_j)
G_H	Computing platform
m	The number of servers
H	A set of all servers
h_i	The i-th server
E_H	A set of links in G_H
(h_i, h_j)	The link between server h_i and server h_j
C	A set of available CPU for all servers
c_i	The available CPU of server h_i
B	A set of bandwidths for all links
b_{ij}	The bandwidth of link (h_i, h_j)
$EST(t)$	The earliest start time of task t without considering server's capacity
$precede(t)$	All preceding tasks of task t
$AT(t_i, t_j)$	Arriving time of data generated from preceding task t_i
$AFT(t)$	Actual finish time of task t
$AST(t)$	Actual start time of task t
$Trans(t_i, t_j)$	Data transmission time
$host(t)$	The server processing task t in a scheduling plan
$AvailC(h, time)$	The available CPU of server h at $time$
$AllCpu(h)$	All CPU on server h
$UsedC(h, time)$	CPU that is already used at $time$
$x(t, h)$	A binary indicator if task t is mapped to server h
$SufC(h, t)$	The time when CPU is sufficient to process task t on server h
FFT	The final finish time of a job under a scheduling plan

3.4 Computational complexity analysis

We prove LRW-TS to be NP-complete in this section. We first introduce the set partition problem, which is included in Karp's 21 NP-complete problems [42], and then reduce it in polynomial time to LRW-TS.

Set partition problem The set partition problem divides a dataset into two subsets such that the sum of each subset is the same. Let L be a dataset and A be a subset of L with sum L_1. The question asks if there exists another subset containing the remainder $(L - A)$ of the elements with sum L_2, and $L_1 = L_2$.

Theorem 1 LRW-TS is NP-complete.

Proof The decision version of LRW-TS is shown as follows: Given a DAG-structured job $G_T = \{T, E_T, W, M, D\}$, a server network $G_H = \{H, E_H, C, B\}$, and a performance bound l, does there exist a scheduling plan such that $FFT \leqslant l$?

We first show that FFT can be calculated in polynomial time. For a given DAG, we perform topological sorting to find a topological order of vertices, which can be done in linear time [43]. From a given starting vertex, we find longest paths in linear time by processing the vertices in a topological order and calculating the path length for each vertex to be the maximum distance via any of its incoming edges [44]. FFT is then obtained by choosing the longest path from all entry nodes in polynomial time.

Given any scheduling plan, we can calculate FFT in polynomial time and then compare it with l to find the correct answer. Therefore, LRW-TS $\in$ NP.

To show the NP-hardness of LRW-TS, we first construct a special case of LRW-TS with a particular network structure, as shown in Fig. 3. There are $n - 1$ entry nodes and one exit node, and two identical servers on the computing platform. We suppose that the running time of the exit node and the data transfer time of all edges are extremely small so that they can be ignored. In this special case, the task dependencies are reduced to a star-shaped DAG, and the network topology is simplified to two homogeneous servers without bandwidth constraints. This simplification removes the influence of network delays and data dependencies, allowing us to focus solely on the task placement problem under an abstract model. While this does not reflect the full complexity of practical LRW-TS scenarios, it enables us to prove that even in this highly restricted version, the problem remains

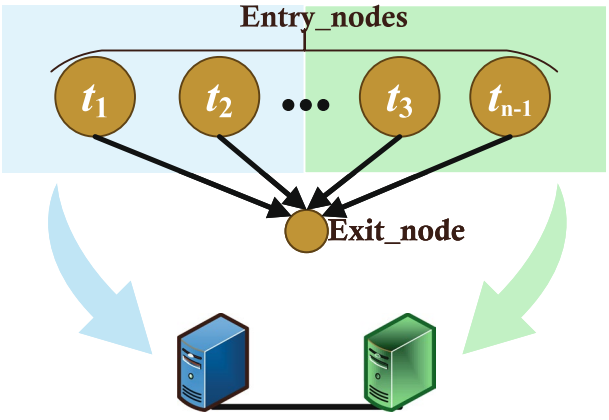


Fig. 3 An instance of LRW-TS with a particular network structure

computationally hard. The objective to minimize the finish time of the job is equivalent to divide the running times of all entry nodes into two subsets such that the sum of one subset is the same as the other. This special case of LRW-TS is actually the set partition problem [42] where the set of L corresponds to the set of running times for all entry nodes, A corresponds to the set of running times of tasks scheduled for execution on one of the servers, and $(L - A)$ corresponds to the set of running times of the remaining tasks. This reduction demonstrates that LRW-TS is at least as hard as the set partition problem. Since this transformation is achievable in polynomial time, and LRW-TS belongs to NP, the problem is NP-complete. □

4 Algorithm design

As proved NP-complete in Sect. 3.4, LRW-TS cannot be solved by a polynomial-time optimal algorithm unless $P = NP$. In this section, we focus on the design of running time gap strategy (RTGS) inspired by the whale optimization algorithm (WOA) [7]. We begin this section by providing an overview of the whale optimization algorithm, followed by an analysis of the positive correlation between the two key variables. Building upon the WOA and this observed relationship, we subsequently design the RTGS scheduling algorithm. RTGS is divided into three parts deriving from humpback whale hunting behaviors. Humpbacks that represent scheduling plans approach the prey, which is the optimal scheduling plan, step by step by choosing one movement from the three-part mechanism.

4.1 Overview of Whale Optimization Algorithm

The whale optimization algorithm (WOA), inspired by the social hunting behavior of humpback whales, particularly the bubble-net feeding strategy, is a nature-inspired metaheuristic. The algorithm simulates both exploration and exploitation phases, which correspond to global and local search processes, respectively. By employing distinct mathematical models for each phase, it achieves a dynamic balance between broad solution space exploration and fine-grained local optimization.

The WOA models the spiral bubble-net attacking mechanism and encircling prey behavior through the following key steps:

- *Encircling prey* Whales recognize the position of the prey and surround it. The encircling mechanism is modeled as:

$$\vec{Dis} = |\vec{Coe} \cdot \vec{X}^*(t) - \vec{X}(t)|, \tag{13}$$

$$\vec{X}(t+1) = \vec{X}^*(t) - \vec{Ac} \cdot \vec{Dis}, \tag{14}$$

where $\vec{X}^*(t)$ is the position vector of the current best solution, $\vec{X}(t)$ is the position vector, and $\vec{Ac}$ and $\vec{Coe}$ are coefficient vectors calculated as:

$$\vec{Ac} = 2\vec{a}' \cdot \vec{r}_1 - \vec{a}', \tag{15}$$

$$\vec{Coe} = 2 \cdot \vec{r}_2, \tag{16}$$

where $\vec{a}'$ linearly decreases from 2 to 0 over iterations and $\vec{r}_1, \vec{r}_2 \sim U(0, 1)$.

- *Bubble-net attacking (exploitation phase)* A spiral-shaped movement simulates whales' bubble-net behavior, enabling local search around promising solutions. This behavior is represented using a spiral equation to update the whale's position:

$$\vec{X}(t+1) = \vec{Dis}' \cdot e^{b \cdot l} \cdot \cos(2\pi l) + \vec{X}^*(t), \tag{17}$$

where $\vec{Dis}' = |\vec{X}^*(t) - \vec{X}(t)|$, b is a constant defining the spiral shape, and l is a random number in $[-1, 1]$.

- *Exploration phase* Whales randomly search for new positions in the global space, ensuring diversity and preventing premature convergence. Whales search randomly for prey if $|\vec{Ac}| > 1$, and the mechanism is modeled as:

$$\vec{Dis} = |\vec{Coe} \cdot \vec{X}_{rand} - \vec{X}(t)|, \tag{18}$$

$$\vec{X}(t+1) = \vec{X}_{rand} - \vec{Ac} \cdot \vec{Dis}, \tag{19}$$

where $\vec{X}_{rand}$ is a randomly selected whale position from the current population.

The general procedure of the WOA begins with initializing a population of whales with random positions in the search space. In each iteration, the fitness values of the candidate solutions are computed using a problem-specific objective function. Subsequently, whales update their positions by either approaching the current best solution or exploring new regions based on a probabilistic criterion. The algorithm iterates until a predefined stopping condition is met, such as the maximum number of iterations or convergence tolerance.

Task scheduling in distributed computing environments presents a high-dimensional combinatorial optimization challenge. It involves efficiently mapping diverse tasks to heterogeneous resources under multiple constraints, including execution time, resource utilization, and data locality. Addressing these complexities requires an optimization algorithm capable of both global exploration and local exploitation. The WOA offers an effective solution to this problem. Its mathematical design enables a dynamic transition between exploration and exploitation phases, which not only improves the global search capability but also enhances local refinement. This balance is crucial for avoiding local optima and identifying near-optimal scheduling strategies in a vast solution space. Additionally, the WOA's solution representation, where each whale's position corresponds to a task-to-resource mapping, supports natural encoding for scheduling problems. Compared with other population-based methods such as genetic algorithm (GA) and particle swarm optimization (PSO), the WOA requires fewer control parameters and simpler update equations, resulting in lower

computational overhead and faster convergence. Moreover, its flexibility allows a seamless integration of task-specific heuristics, including urgency awareness, data locality constraint, and task dependency handling. Considering these advantages, we adopt the WOA as the core optimization engine in this study. To further align with the requirements of large-scale data processing systems, we introduce domain-specific enhancements to improve scheduling effectiveness. These enhancements are discussed in detail in the following subsections.

4.2 Positive correlation analysis

In this section, we reveal a positive relationship between two variables through empirical study, i.e., scheduling plan distance (*SPD*) and finish time gap (*FTG*) under two different scheduling plans.

Scheduling plan distance (SPD) is the distance between two scheduling plans $\mathcal{S}_1$ and $\mathcal{S}_2$, calculated as

$$SPD = \frac{1}{n} \sum_{t \in T} \left| load_{\mathcal{S}_1}(host(t)) - load_{\mathcal{S}_2}(host(t)) \right|, \tag{20}$$

where n is the number of tasks, $load_{\mathcal{S}_i}(host(t))$ is the sum of running times of all tasks deployed on the host processing task t in scheduling plan $\mathcal{S}_i$, and $i = 1$ or 2.

For a given task t, we first calculate the gap between the running times of two task groups from two scheduling plans, where each task group contains the tasks running on the same server processing task t. We then calculate SPD as the average of all absolute gaps from all tasks.

Finish time gap (*FTG*) is the absolute difference of the final finish times of two scheduling plans, calculated as

$$FTG = \left| FFT(\mathcal{S}_1) - FFT(\mathcal{S}_2) \right|, \tag{21}$$

where $FFT(\mathcal{S}_i)$ is the final finish time for scheduling plan $\mathcal{S}_i$, $i = 1$ or 2.

In RTGS, we use *FTG* to measure the distance between two scheduling plans as it provides a straightforward understanding of the difference between two scheduling plans. However, the *FTG* metric cannot be directly used to generate a new scheduling plan. To address this issue, we investigate the relationship between *SPD* and *FTG*, and reveal below a positive relationship between them. Therefore, we are able to manipulate *FTG* through *SPD*. This is an important insight because we can use the function of *SPD* for derivation to generate a new scheduling plan as detailed in Sect. 4.3.3.

To analyze the relationship between *SPD* and *FTG* in depth, we examine a specific workload case with a particular network structure, as shown in Fig. 3, where the execution time for each entry node is 1 s. Initially, the scheduling plan $\mathcal{S}_1$ assigns each host to execute $(n-1)/2$ entry tasks. In the alternative scheduling plan $\mathcal{S}_2$, host h_1 executes n_1 entry tasks, while host h_2 executes n_2 entry tasks, where $n_1 > (n-1)/2$ and $n_2 < (n-1)/2$. Let $num' = n_1 - (n-1)/2$. We then have:

Table 3 Pearson correlation coefficients

Number of tasks	100	200	300	400	500	600	700
PCC in busy time	0.4147	0.4487	0.3732	0.5243	0.4233	0.4536	0.4931
PCC in idle time	0.4192	0.1627	0.3724	0.3622	0.4153	0.3532	0.4133
Number of tasks	800	900	1000	2000	3000	4000	5000
PCC in busy time	0.4638	0.3632	0.4074	0.4632	0.4306	0.4173	0.4834
PCC in idle time	0.4690	0.3990	0.4913	0.4145	0.4905	0.4104	0.4794
Number of tasks	6000	7000	8000	9000	10,000		
PCC in busy time	0.3482	0.5006	0.3672	0.4620	0.3210		
PCC in idle time	0.3849	0.5374	0.4412	0.3756	0.3205		

$$SPD = num' \cdot \left(num' + \frac{n-1}{2}\right) + num' \cdot \left(\frac{n-1}{2} - num'\right), \tag{22}$$

$$SPD = num' \cdot (n-1), \tag{23}$$

$$FTG = num'. \tag{24}$$

According to Eqs. 23 and 24, we have:

$$SPD = (n-1) \cdot FTG, \tag{25}$$

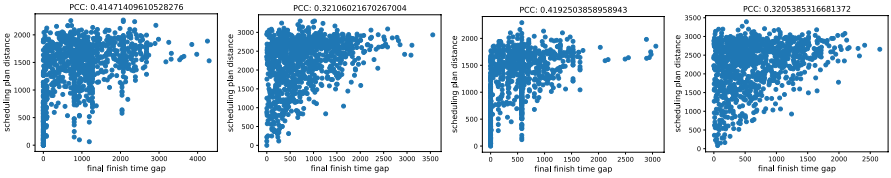
which demonstrates the linear correlation between *SPD* and *FTG* in this specific scenario.

Furthermore, we conduct a large number of experiments based on the dataset from Alibaba Cluster Trace Program[2] to investigate the relationship between *SPD* and *FTG*. The trace is sampled from Alibaba production clusters. Cluster-trace-2018 from Alibaba Cluster Trace Program includes about 4000 machines over a period of 8 days. This piece of trace contains DAG information of their production batch workloads and online services (a.k.a. long-running applications). The analysis is composed of two groups of experiments according to server busy and idle time, respectively. Jobs are selected randomly from the trace with different computing scales, and the number of tasks increases from 100 to 1000 and from 1000 to 10, 000. All parameters are provided in Table 4.

We calculate Pearson correlation coefficient (PCC) [45] in each group of experiments in Table 3, which shows that *SPD* has a medium positive correlation with *FTG* [46]. The values of PCC range from 0.16 to 0.53, and the average value of PCC is 0.4177. For illustration, we choose four experiments to draw scatter plots with 100 and 10, 000 tasks as shown in Fig. 4. The Python code and data for producing these plots can be found in the GitHub repository.[3] Figure 4 shows that the value of *SPD* is more likely to increase with a larger *FTG*. This is an important feature that guides the design of our algorithm for a better search performance.

[2] https://github.com/alibaba/clusterdata.

[3] https://github.com/AIprinter/task-scheduling.



(a) task number:100, busy (b) task number:1.0e+04, busy (c) task number:100, idle (d) task number:1.0e+04, idle

Fig. 4 Illustration of PCC between *SPD* and *FTG*

Table 4 Parameters in the experiments

Parameters	Value
Repeat times of each group of experiments	1000
Number of servers	50
Number of tasks in small-scale computing	[100, 200, … , 1000]
Number of tasks in large-scale computing	[1000, 2000, … , 10000]
CPU range in busy time	[200, 500]
CPU range in idle time	[200, 2000]

4.3 WOA-based scheduling algorithm: RTGS

We design RTGS based on WOA, which is a novel nature-inspired metaheuristic optimization algorithm proposed in 2016. Seyedali et al. [7] designed WOA based on humpback whale's hunting behavior [47]. In RTGS, a humpback corresponds to a scheduling plan represented by a vector of n dimensions, and each dimension is a server number, on which the task is assigned to run. There are a fixed number of humpback whales in each iteration. Following the process of WOA, we divide RTGS into three parts: encircling prey, spiral bubble-net feeding maneuver, and search for prey. In each iteration, every humpback moves according to one of these three parts, as detailed below.

4.3.1 Encircling prey

Humpbacks can locate preys and encircle them. The location of a prey corresponds to the optimal scheduling plan (OSP) in LRW-TS. However, since OSP is not known in advance, RTGS considers the current best scheduling plan, which is selected as the leader of all humpbacks, to be the target prey in each iteration. When the leader humpback is selected, other humpbacks would update their locations toward the leader, modeled as:

$$D = \left| C \cdot \mathcal{S}^*(k) - \mathcal{S}(k) \right|, \tag{26}$$

$$\mathcal{S}(k+1) = \mathcal{S}^*(k) - A \cdot D, \tag{27}$$

where k is an iteration number, A and C are coefficients, $\mathcal{S}^*(k)$ indicates the leader location, $\mathcal{S}(k)$ is a humpback location in iteration k, $\mathcal{S}(k+1)$ is the next-step location of humpback $\mathcal{S}(k)$, $|\,|$ takes the absolute value of each element, operator $\cdot$ performs element-by-element multiplication, and D represents the distance from $\mathcal{S}(k)$ to $\mathcal{S}^*(k)$. Coefficients A and C are defined as follows:

$$A = 2a \cdot r - a, \tag{28}$$

$$C = 2 \cdot r - 1, \tag{29}$$

$$a = 2 \cdot \left(1 - \frac{k}{k_{\max}}\right), \tag{30}$$

where r is a random value in $[0, 1]$, and $k_{\max}$ is the maximum iteration number. As the iteration increases, the value of a decreases from 2 to 0. Since A is in the range of $[-a, a]$, the value of A has a tendency to decrease as the iteration proceeds, which is attributed to the decrease in the value of a.

In the real world, we can use Eq. (26) to calculate the distance between different humpbacks. However, if we use the same equation to calculate the distance between different scheduling plans, it would not make much sense. Note that each dimension in a scheduling plan is a server number. One server number minus another does not carry much physical meaning. To address this issue, we introduce the concept of *FTG*, which serves as a metric for measuring the distance between two humpbacks, calculated using Eq. (21). However, it is not straightforward to directly utilize *FTG* to generate new humpback locations or scheduling plans in each iteration. To overcome this limitation, we leverage the positive relationship between *FTG* and *SPD* as unveiled in Sect. 4.2. By transforming *FTG* to *SPD*, we obtain a more practical quantity that can be employed to generate new humpback locations or scheduling plans during the iterative process.

According to a particular *FTG*, it is impossible to create a new next-step humpback location through Eq. (27), which means that $\mathcal{S}(t+1)$ cannot be obtained from Eq. (27) based on the value of *FTG* calculated using Eq. (21). The reason is that *FTG* is a constant value and scheduling plans are represented as multidimensional vectors. We need to find a specific method or function to express *FTG*. This function should be able to extend into a multidimensional space to align with the complexity of scheduling plans. To address this issue, we leverage our finding about the medium positive correlation between *FTG* and *SPD*: *FTG* increases as the value of *SPD* increases. If *SPD* is controlled, so is *FTG*. Then, we can replace D with *SPD* to estimate the humpback's next-step location. The other key point about *SPD* is that *SPD* can be extended into a multidimensional space. This extension allows us to leverage *SPD* in the derivation process, enabling the creation of a specialized function to generate new scheduling plans. We integrate this insight with a greedy algorithm into the design RTGS, which enables us to control each step of the optimal scheduling plan hunting process effectively.

At the end of the derivation, we obtain a series of hope values represented by $hope(t_i)$, which is equal to $load_{\mathcal{S}_{(k+1)}}(host(t_i))$, which is the sum of the running times

of all tasks located on the server processing task t_i at the humpback's next-step location $\mathcal{S}(k+1)$. To reassign each task and find the humpback's next-step location, we need first obtain the hope value. We derive the hope value for each task at $\mathcal{S}(k+1)$ according to the leader location in this iteration and the current location of a humpback. We use $lead(t_i)$ to represent the information of the leader scheduling plan, and use $curr(t_i)$ to represent a normal scheduling plan's information in the current iteration.

$$hope(t_i) = load_{\mathcal{S}_{(k+1)}}(host(t_i)), \tag{31}$$

$$lead(t_i) = load_{\mathcal{S}^*_{(k)}}(host(t_i)), \tag{32}$$

$$curr(t_i) = load_{\mathcal{S}_{(k)}}(host(t_i)). \tag{33}$$

According to Eqs. (21), (26), and (27), we have

$$\begin{aligned} \mathcal{S}^*(k) - \mathcal{S}(k+1) &= A \cdot \left| C \cdot \mathcal{S}^*(k) - \mathcal{S}(k) \right|, \\ FTG\left(\mathcal{S}^*(k), \mathcal{S}(k+1)\right) &= A \cdot FTG\left(\mathcal{S}^*(k), \mathcal{S}(k)\right), \end{aligned} \tag{34}$$

where D is replaced with FTG to measure the distance between two scheduling plans. We observe that the variable C disappears in this step of the derivation process. This variable is responsible for introducing randomness to the leader whale's position. By setting $C = 1$, the leader's position remains fixed.

Given the positive correlation between SPD and FTG, as described in Eq. (20), we substitute the value of SPD for FTG in Eq. (34). Then, we have

$$\begin{aligned} \frac{1}{n} \sum_{t_i \in T} \left| lead(t_i) - hope(t_i) \right| &= \frac{A}{n} \sum_{t_i \in T} \left| lead(t_i) - curr(t_i) \right|, \\ \sum_{t_i \in T} \left| lead(t_i) - hope(t_i) \right| &= \sum_{t_i \in T} A \cdot \left| lead(t_i) - curr(t_i) \right|. \end{aligned}$$

In the derivation of the above equations, we suppose that every two corresponding items are equal to each other:

$$\left| lead(t_i) - hope(t_i) \right| = A \cdot \left| lead(t_i) - curr(t_i) \right|.$$

The hope value of $load_{\mathcal{S}_{(k+1)}}(host(t_i))$ is derived as:

$$\begin{aligned} lead(t_i) - hope(t_i) &= \pm A \cdot \left(lead(t_i) - curr(t_i) \right), \\ hope(t_i) &= lead(t_i) \pm A \cdot \left(lead(t_i) - curr(t_i) \right). \end{aligned} \tag{35}$$

In RTGS, each humpback selects a direction to step in. In each search direction, we perform a derivation process to create the humpback's next-step location. During this derivation process, we obtain different functions for calculating the value of $hope(t_i)$. Once a search direction is selected, RTGS utilizes the value of

$hope(t_i)$ obtained from the corresponding function. This value serves as input for a specific greedy algorithm, which generates the next humpback location based on the optimization goal.

4.3.2 Bubble-net attacking method (exploitation phase)

Humpbacks' special hunting method is referred to as bubble-net feeding [48]. Humpbacks dive about 12 m down, and then start to create bubbles on a '9'-shaped path around the school of fish and swim up toward the prey. Humpbacks' bubble-net attacking behavior is divided into two parts: shrinking encircling mechanism and spiral updating position.

Shrinking encircling mechanism In Eq. (27), A is a random value in $[-a, a]$, where a is decreased as the iteration increases, and the value of A is decreased with iteration variable k. In other words, the preying circle is shrinking gradually due to the decreasing value of A.

Spiral updating position Seyedali et al. [7] provided a spiral equation to simulate the spiral-shaped path of humpbacks. The equation first calculates the distance between a humpback located at $\mathcal{S}(k)$ to the leader humpback located at $\mathcal{S}^*(k)$. A spiral location is created between $\mathcal{S}(k)$ and $\mathcal{S}^*(k)$ based on the distance calculated using the following equation:

$$\mathcal{S}(k+1) = D' \cdot e^{bl} \cdot \cos(2\pi l) + \mathcal{S}^*(k), \tag{36}$$

where $D' = \left|\mathcal{S}^*(k) - \mathcal{S}(k)\right|$ is the distance from a humpback's current location to the leader humpback, b is a constant, and l is a random value between -1 and 1.

As in encircling a prey, to find the humpback's next step when it swims as a spiral shape, we first need to derive the hope value of $load_{\mathcal{S}_{(k+1)}}\left(host(t_i)\right)$ at the humpback's next step $\mathcal{S}(k+1)$. With the same derivation as in the phase of encircling a prey, we calculate the hope value in this phase as:

$$hope(t_i) = lead(t_i) \pm A' \cdot \left(lead(t_i) - curr(t_i)\right), \tag{37}$$

$$A' = e^{bl} \cdot \cos(2\pi l). \tag{38}$$

Note that humpbacks shrink the circle and swim along a spiral-shaped path in a simultaneous way. In order to mimic this simultaneous behavior, we use a variable p to determine which behavior should a humpback go through, where p is a random value in $[0, 1]$. The humpback updates its location according to the shrinking encircling mechanism if $p \leq 0.5$, or updates its location based on spiral updating:

$$hope(t_i) = \begin{cases} lead(t_i) \pm A \cdot \left(lead(t_i) - curr(t_i)\right), & p \leqslant 0.5 \\ lead(t_i) \pm A' \cdot \left(lead(t_i) - curr(t_i)\right), & p > 0.5. \end{cases} \tag{39}$$

4.3.3 Search for prey (exploration phase)

In a real-life humpback hunting process, humpbacks have the knowledge of the location of fish school, and hence swim directly toward their goal. However, in the search process of a suboptimal scheduling plan, all humpbacks move toward a temporary optimal solution, which may lead to a local optimum. To achieve a global optimal solution, humpbacks are pushed away when $|A| > 1$ to search for other better scheduling plan. In this case, the leader humpback is replaced with a randomly selected humpback, and the humpback located at $\mathcal{S}(k)$ is pushed away by the randomly selected humpback whale. This process is modeled mathematically as:

$$\mathcal{S}(k+1) = \mathcal{S}_{rand} - A \cdot D, \tag{40}$$

$$D = \left| C \cdot \mathcal{S}_{rand} - \mathcal{S}(k) \right|, \tag{41}$$

where $\mathcal{S}_{rand}$ is a randomly selected humpback whale.

The hope value of $load_{\mathcal{S}_{(k+1)}}\left(host\left(t_i\right)\right)$ for the humpback, which is pushed away for searching the global optimal solution, is derived as:

$$hope\left(t_i\right) = rand\left(t_i\right) \pm A \cdot \left(rand\left(t_i\right) - curr\left(t_i\right)\right), \tag{42}$$

where $rand\left(t_i\right)$ is equal to $load_{\mathcal{S}_{rand}}\left(host\left(t_i\right)\right)$, which is the sum of all task running time located on the server processing task t_i in the randomly selected humpback.

So far, we have described the three mechanisms involved in the humpback whale's hunting behavior, and obtained three corresponding lists of hope values, $\left\{hope\left(t_i\right) | t_i \in T\right\}$, each representing the outcomes of the respective mechanism for guiding the whale's next move. Next, we need to create the next-step location of humpbacks using the list of $hope\left(t_i\right)$. RTGS uses a greedy algorithm to satisfy the list of requirements $\left\{hope\left(t_i\right) |\ t_i \in T\right\}$ to locate the humpback's next step.

4.3.4 Greedy algorithm for generating new scheduling schemes

In the previous sections, we described three mechanisms inspired by the hunting behavior of humpback whales, each generating a list of hope values $\left\{hope(t_i) | t_i \in T\right\}$ that guide the whale's movement toward optimal scheduling decisions. Building upon this foundation, we now introduce a greedy algorithm that generates the new scheduling plans based on the list of hope values $\left\{hope(t_i) | t_i \in T\right\}$. This algorithm is designed to refine the scheduling scheme by selectively reassigning a portion of tasks based on their *hope* values. By balancing exploitation and exploration through adaptive parameters and local estimation of server loads, the greedy algorithm helps iteratively approach a more efficient task-to-server allocation.

Algorithm 1 Greedy algorithm to find a humpback's next step

Input: $\mathcal{S}^*(k)$, $\mathcal{S}(k)$, p_n, A, p;
$\mathcal{S}(k+1) \leftarrow \mathcal{S}^*(k)$;
Choose randomly $p_n \cdot a$ percent of tasks as $list$;
In $\mathcal{S}(k+1)$, $x(t,h) = -1$ if $t \in list$;
$THost_j \leftarrow \sum_{t \in T} w_t \cdot x\left(t, h_j\right)$, $j \in H$ for $\mathcal{S}(k+1)$;
$TList \leftarrow \sum_{t \in list} w_t$ for $\mathcal{S}(k+1)$;
$TAll \leftarrow \sum_{t \in T} w_t$ for $\mathcal{S}(k+1)$;
for $t_i \in list$ **do**
    Update $hope(t_i)$ according to Eqs. (35), (37), or (42);
    $EV\left(h_j\right) \leftarrow \left(THost_j + w_{t_i}\right) / \left(TAll - TList + w_{t_i}\right) * TAll$;
    Set $x(i,k) = 1$ if $|EV(h_k) - hope(t_i)|$ is the smallest among all servers.
    $THost_{AssignedHost} \leftarrow THost_{AssignedHost} + w_{t_i}$;
    $TList \leftarrow Tlist - w_{t_i}$;
    $list \leftarrow list - t_i$;
Output: $x_{n \times m}$.

The pseudocode of this greedy algorithm is provided in Algorithm 1. As humpbacks approach the fish school, their swimming speed slows down. In the greedy algorithm, we choose $p_n \cdot a$ percent of tasks waiting for relocation to mimic the humpback's gradually decreased step as it approaches a prey. Note that p_n is a hyper-parameter in [0, 1], and the value of a decreases gradually as the iteration k increases as shown in Eq. (30). The exploitation and exploration phases may have different values of p_n, because the exploration phase needs a bigger step to search for the global optimal solution. The tasks waiting for relocation are selected randomly. Other tasks that are not selected have the same processor as the leader humpback $\mathcal{S}^*(k)$ or the random humpback $\mathcal{S}_{rand}$ selected in the exploration phase. In the greedy algorithm, we need to estimate which server has the closest value to the required $hope\left(t_i\right)$, and then assign task t_i to the server for execution. It is worth mentioning here that at the beginning of the greedy algorithm, $\left(1 - p_n \cdot a\right)$ percent of tasks have been assigned to its servers for execution, and $\left(1 - p_n \cdot a\right)$ percent of tasks have the same server as the leader humpback or a random selected humpback. The other $p_n \cdot a$ percent of tasks are the set of tasks waiting for reassignment. We estimate the load on each server of the humpback's next step according to current $\mathcal{S}^*(k+1)$ obtained so far. Note that only part of tasks in $\mathcal{S}^*(k+1)$ have been mapped to a server. We estimate all tasks running time in a server of humpback next step proportionally using Eq. (43). The real summation of the running time of all tasks located on a server running task t_i at the humpback's next step $\mathcal{S}(k+1)$ is estimated using Eq. (43), which is referred to as Estimated Value (EV):

$$EV\left(t_i, h_j\right) = \left(\sum_{t \in T} w_t \cdot x\left(t, h_j\right) + w_{t_i}\right) / \left(1 - \frac{\sum_{t \in (list - t_i)} w_t}{\sum_{t \in T} w_t}\right), \tag{43}$$

where *list* is the list of tasks waiting for reassignment. The task location is completed one by one. Once one task is completed, it is removed from the task list *list*. $x\left(t, h_{\mathrm{j}}\right)$ is based on the current state of $\mathcal{S}^{*}(k+1)$ where not all tasks have been scheduled. The greedy algorithm chooses the server, which has the closest $EV\left(t_{\mathrm{i}}, h_{\mathrm{j}}\right)$ to its $hope\left(t_i\right)$ to execute the task t_i.

To provide a clearer illustration of the logical structure among the four modules of the RTGS algorithm: encircling prey, bubble-net attacking method, search for prey, and greedy algorithm for generating new scheduling schemes, the workflow of RTGS algorithm is depicted in a flowchart shown in Fig. 5. As shown in Fig. 5, the RTGS process begins with population initialization, where each whale position represents a potential scheduling plan. Then, the fitness of each plan is evaluated to determine the current best solution $\mathcal{S}^{*}$. Afterward, key parameters including a, A, l, p, and p_n are dynamically updated to guide the behavior of the whales in subsequent steps. Depending on the value of p, the algorithm transitions into either the exploration or exploitation phase. When $p \geq 0.5$, the algorithm enters the exploitation phase and updates the hope list $hope(t_i)$ using the bubble-net attacking mechanism. Conversely, if $p < 0.5$, it switches to the exploration phase, where the algorithm chooses between searching for prey or encircling prey, based on the magnitude of $|A|$. With the updated hope list, the greedy algorithm is employed to generate a new scheduling plan by selectively reassigning tasks. This iterative process continues until the maximum number of iterations $k_{\max}$ is reached, at which point the algorithm outputs the best scheduling plan $\mathcal{S}^{*}$.

Algorithm 2 RTGS Integrated Code

Input: p_{n1} and p_{n2} for exploitation and exploration phases respectively, maximum iterations *numIter*, number of humpbacks *numHump*;
Randomly initialize all humpbacks $\mathcal{S}_{\mathrm{i}}$ $(i = 1, 2, ..., num)$;
Calculate FFT of each humpback according to Eq. (12);
Assign the humpback with the smallest FFT as leader $\mathcal{S}^{*}$;
$k \leftarrow 0$;
while $k <$ *numIter* **do**
  for each humpback **do**
    Randomly generate values for r and p;
    Update A according to Eq. (28);
    if $p < 0.5$ **then**
      if $|A| < 1$ **then**
        $p_n \leftarrow p_{n1}$;
        Update humpback according to Alg. 1;
      if $|A| \geqslant 1$ **then**
        $p_n \leftarrow p_{n2}$;
        $\mathcal{S}^{*} = \mathcal{S}_{rand}$;
        Update humpback according to Alg. 1;
    else if $p \geqslant 0.5$ **then**
      $A = e^{bl} \cdot \cos(2\pi l)$;
      Update humpback according to Alg. 1;
  Update FFT for each humpback;
  Update $\mathcal{S}^{*}$;
  $k = k + 1$;
Output: $\mathcal{S}^{*}$.

Table 5 Parameters in the algorithm design

Parameters	Definition
SPD	The distance between two scheduling plans
k	The iteration number
$k_{\max}$	The maximum iteration number
$load_{\mathcal{S}}(host(t))$	The sum of running times of tasks deployed on the host processing task t in scheduling plan $\mathcal{S}$
FTG	The final finish time gap of two scheduling plans
D	The distance of two humpbacks in WOA encircling prey phase
D'	The distance of two humpbacks in WOA spiral updating phase
$\mathcal{S}^*$	The location of the leader humpback
$\mathcal{S}(k)$	The humpback location in iteration k
r, l	Two random numbers
$hope(t)$	$load_{\mathcal{S}_{(k+1)}}(host(t))$
$lead(t)$	$load_{\mathcal{S}^*_{(k)}}(host(t))$
$curr(t)$	$load_{\mathcal{S}_{(k)}}(host(t))$
$rand(t)$	$load_{\mathcal{S}_{rand}}(host(t))$
$EV(t_i, h_j)$	The estimate value for the sum of task running times on h_j in the humpback's next step if t_i is mapped to h_j
$list$	A list of tasks waiting for reassignment
p	A random number to determine either a spiral or a circular movement
p_n	A parameter to control the step size
A, C, A', a	Coefficients

The pseudocode of RTGS is provided in Algorithm 2. RTGS starts with a set of randomly initialized humpbacks. All humpbacks approach the prey step by step. In each step, RTGS first determines which humpback is the leader, and then every humpback selects one out of the three mechanisms: shrinking encircling mechanism, spiral updating position, and search for prey independently to swim toward the leader or away from a randomly selected humpback. When a humpback determines its swimming manner, we obtain a list of hope values $\{hope(t_i) \mid t_i \in T\}$. We create a new humpback location by satisfying this list of hope values using a greedy algorithm. If the list of hope values is satisfied, the specific value of FTG is also satisfied due to the positive relationship between FTG and SPD. The value of $|A|$ is decreased gradually from 2 to 0 as the iteration proceeds in order to perform the exploration and exploitation phases. RTGS chooses either a spiral or a circular movement depending on a random parameter p. The greedy algorithm is proposed to create the humpback's next step with respect to the final finish time gap of different humpbacks. The parameter p_n is used to control the step size manually in different search phases. RTGS ends when reaching the maximum number of iterations. For convenience, we tabulate in Table 5 the parameters in the algorithm design.

The main computational cost of RTGS lies in the iterative update of each humpback solution, as detailed in Algorithm 2, where Algorithm 1 serves as the core subroutine for position updates. Let n be the number of tasks and m be the number of servers.

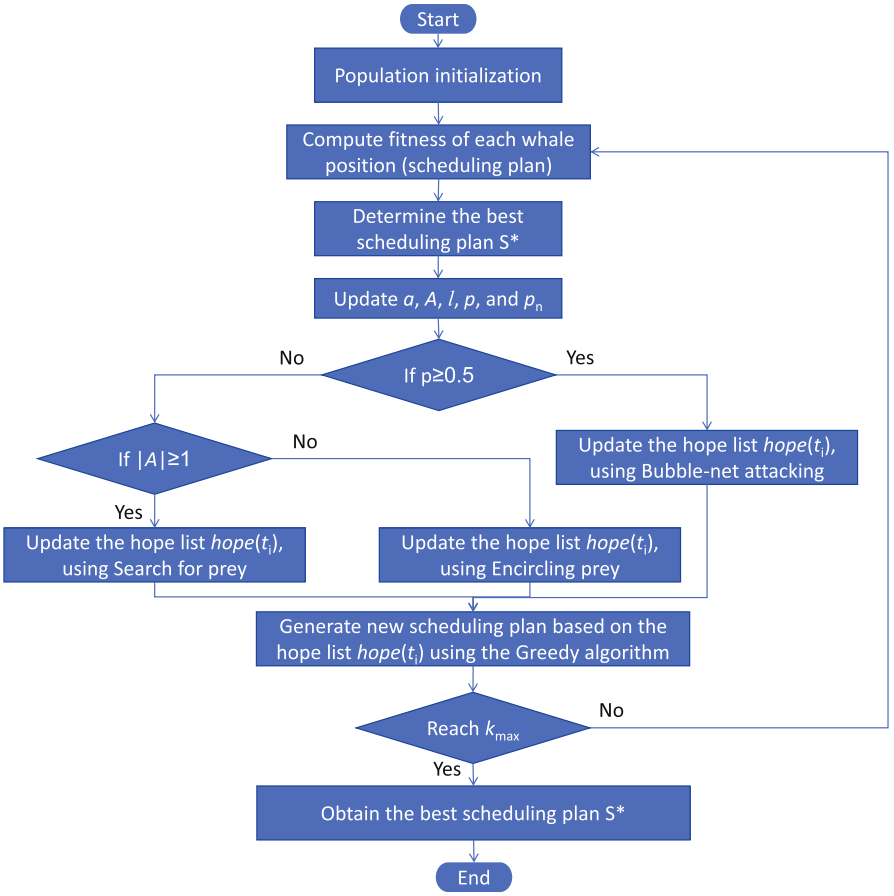


Fig. 5 Flowchart of the RTGS algorithm

Within each call to Algorithm 1, it first selects up to $O(n)$ candidate tasks in lines 3–4. For each selected task t_i, the following operations are performed: line 9 calculates $hope(t_i)$, which depends on the equations involving all tasks or servers, taking $O(n)$ time; line 10 iterates over all m servers to compute the expected value deviation, taking $O(m)$ time. Hence, the time complexity of one invocation of Algorithm 1 is $O(n^2 + nm)$. This reflects a polynomial-time algorithm in terms of the number n of tasks and the number m of servers. While the computational cost increases with problem size, RTGS avoids the exponential complexity associated with exact methods and remains scalable for large-scale long-running workloads, as demonstrated in our experimental evaluation.

5 Performance evaluation

5.1 Experimental setup

Alibaba Cluster Trace v2018[4] is sampled from Alibaba production clusters and includes about 4000 machines over a period of 8 days. This trace contains DAG-structured workloads, machine states at every time stamp, and detailed information about workload execution. To represent short- and long-running workloads, we randomly sample jobs from the trace with 100–1000 subtasks and 1000–10,000 subtasks, respectively. It is worth mentioning that the number of tasks is a rough estimate. The evaluation is composed of two groups of experiments based on workloads with DAG structures and workloads with independent tasks. Different server states during busy and idle periods, respectively, are considered for the experiments.

We implement the proposed RTGS and other comparison scheduling algorithms in Java, and run workload scheduling experiments on a cluster of nine local servers installed with CentOS Linux 7 (Core). Among them, six servers are equipped with an Intel(R) Xeon(R) E5-2620 CPU (2.10GHz), 32GB DDR4 RAM, and 600GB SSD disk for storage, two servers are equipped with an Intel(R) Xeon(R) E5-2620 CPU (2.10GHz), 16GB DDR4 RAM, and 260GB SSD disk for storage, and one server is equipped with an Intel(R) Xeon(R) E5-2620 CPU (2.10GHz), 32GB DDR4 RAM, and 170GB SSD disk for storage. The experimental results are processed and visualized using Python on a Windows PC. All source codes, experimental data, and selected instances can be found in the GitHub repository.[5]

5.2 Comparison algorithms

We evaluate the performance of RTGS in comparison with seven state-of-the-art metaheuristic approaches. IWC [49] is a WOA-based method aiming at multi-objective optimization such as reducing resource usage and operational cost of the system. WOA [7], ACO [50], PSO [51], and GA [52] are well-known bioinspired, randomization-based algorithms used to solve optimization problems. Additionally, we conduct comparative experiments with the multi-objective Harris hawks optimization-based algorithm (MoHHOTS) [26] and the graph-oriented simulated annealing (GOSA) algorithm [27], which have shown promising results in similar contexts. The main implementation parameters used in our experiments for each algorithm are provided in Table 6. All algorithms are initialized with 50 individuals and terminated with the fixed maximum iteration number of 20. First, we demonstrate how these parameters are chosen through our empirical study, and then present the *FFT* comparison results among all algorithms. The Java code for reproducing the results can be found in the GitHub repository[5].

[4] https://github.com/alibaba/clusterdata.

[5] https://github.com/AIprinter/task-scheduling.

Table 6 Main parameters used in the experiments

Algorithm	Parameter	Description
ACO	$\rho = 0.7$	Pheromone evaporation coefficient
	$p = 0.3$	Probability of selecting a path
PSO	$\omega = 0.9$	Inertia weight
	$c1 = 1.8$	Acceleration constant
	$c2 = 1.8$	Acceleration constant
IWC	$b = 1$	Defining the shape of logarithmic spiral
	$PS_{\max} = 51$	Largest population
	$PS_{\min} = 10$	Initial population
	$\alpha = 0.25$	Individual generate parameter
	$\gamma = 20$	Nonlinear factor
WOA	$b = 1$	Defining the shape of logarithmic spiral
GA	0.5	Crossover probability
	0.001	Mutation probability
RTGS	$p_n = 0.5$	Parameter value in exploration phase
	$p_n = 0.04$	Parameter value in exploitation phase
	$b = 1$	Defining the shape of logarithmic spiral
	$k_{max} = 20$	Maximum iteration number

Classical metaheuristic algorithms for task scheduling, including GA, PSO, WOA, and ACO, exhibit both time and space complexities of $O(n \cdot m)$, where n and m denote the number of tasks and servers, respectively. These algorithms maintain simplicity and scalability but may lack accuracy in complex or dynamic scheduling environments. The GOSA algorithm achieves a lower time complexity of $O(n \cdot \log n)$, yet it demands a higher space complexity of $O(n^2)$, making it less suitable for memory-constrained systems. MoHHOTS demonstrates a lightweight design with both time and space complexities reduced to $O(n + m)$, attributed to its efficient objective formulation and streamlined encoding structure. However, this simplicity may come at the cost of global search precision. In contrast, IWC presents the highest computational burden with a time complexity of $O(n^3 + m)$ and a space complexity of $O(n^2 + m)$, which can severely affect performance when scaling to larger task sets. The proposed RTGS algorithm achieves a trade-off by adopting a time complexity of $O(n^2 + n \cdot m)$ and a space complexity of $O(n \cdot m)$. This ensures higher precision than linear-complexity methods while maintaining better scalability than cubic or quadratic-space algorithms. The design of RTGS enables it to adapt to system dynamics, preserve scheduling accuracy, and handle large-scale workloads efficiently, thereby offering a practical and computationally balanced solution among all approaches in comparison.

5.3 Parameter sensitivity analysis

In this section, we conduct a comprehensive sensitivity analysis of key parameters involved in the RTGS algorithm, particularly those derived from the WOA and our

Table 7 Parameter settings for sensitivity analysis experiments

Parameter	Range	Step size	Other parameters fixed?	Analysis step
p_{n1}	[0.1, 1.0]	0.1	Yes	Step 1
p_{n2}	[0.01, 0.1]	0.01	Yes	Step 2
b	[0.2, 3.0]	0.2	Yes	Step 3
$k_{\max}$	[1, 70]	1	Yes	Step 4

enhanced strategy. The purpose of this analysis is to investigate how variations in these parameters affect the performance of the algorithm, and to determine suitable values for stable and efficient scheduling outcomes. Specifically, we analyze the parameters p_{n1} and p_{n2}, which are introduced in the improved algorithm based on the WOA to control the position updating behavior during the exploration and exploitation phases, respectively, as well as the parameters A and b, which are fundamental components of the WOA framework. For each parameter, we perform targeted experiments to observe its impact and provide recommendations for its selection.

A sensitivity analysis of parameter A first requires a brief examination of its formulation to identify which variables merit further investigation. As defined in Eq. (28), $A = 2a \cdot r - a$, where r is a uniformly distributed random number in the interval $[0, 1]$, generated dynamically within the RTGS algorithm. The coefficient a is computed using Eq. (30): $a = 2 \cdot (1 - k/k_{\max})$, where k denotes the current iteration number, and $k_{\max}$ is the predefined maximum number of iterations. While k varies automatically during the algorithm's execution and thus requires no sensitivity analysis, $k_{\max}$ is a user-specified parameter that directly affects the value of A. Accordingly, in the context of sensitivity analysis, the focus is placed on evaluating the impact of $k_{\max}$.

To accurately assess the impact of each parameter on the performance of the proposed algorithm, a structured sensitivity analysis is performed in three successive stages. This process adopts the classical principle of varying only one parameter at a time while keeping all others unchanged, thereby isolating the effect of the parameter under investigation. In the first stage, the parameters p_{n1} and p_{n2} are examined, with b fixed at 1 and the maximum iteration count $k_{\max}$ set to be a sufficiently large value of 50 to minimize its influence. In the second stage, the sensitivity of b, which defines the shape of the logarithmic spiral in the WOA mechanism, is analyzed while keeping p_{n1}, p_{n2}, and $k_{\max}$ unchanged. Here, p_{n1} and p_{n2} are set to be the optimal values obtained from the first stage. In the final stage, $k_{\max}$ is evaluated with p_{n1}, p_{n2} and b fixed. The parameter settings used in the sensitivity analysis experiments are summarized in Table 7. This experimental framework ensures a rigorous and reliable evaluation of each parameter's individual contribution to the algorithm's performance.

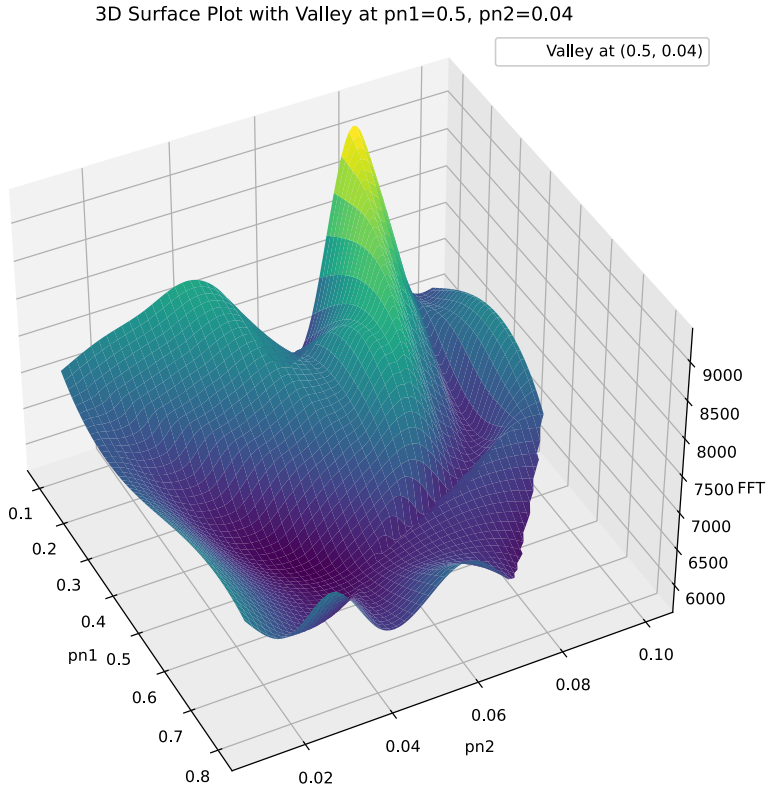


Fig. 6 3D surface plot of average *FFT* measurements for different (p_{n1}, p_{n2}) pairs

5.3.1 Sensitivity analysis of parameter p_n

We use p_{n1} to represent p_n in the exploration phase and p_{n2} to represent p_n in the exploitation phase. We conduct a series of experiments to determine the values of p_{n1} and p_{n2}, where p_{n1} ranges from 0.1 to 1.0 and p_{n2} ranges from 0.01 to 0.1, resulting in 100 pairs of (p_{n1}, p_{n2}). For each pair, we calculate the average *FFT* value across the 19 short- and long-running workloads during the busy period. These average *FFT* values are used to create a 3D surface plot using the cubic interpolation algorithm [53] as shown in Fig. 6. The valley of the 3D surface plot is observed near the point of (0.5, 0.04).

Interestingly, the 3D surface plot reveals a performance dip when both p_{n1} and p_{n2} are set to intermediate values. This decline may result from a weakened balance between exploration and exploitation. When both parameters are moderate, the algorithm neither explores sufficiently diverse regions in the early stages nor performs focused refinement in the later stages. Consequently, the algorithm lacks a clear phase distinction, and the search behavior becomes

indecisive, which could lead to premature convergence in suboptimal regions and ultimately result in a longer average job completion time.

In addition, the 3D surface plot shows that the algorithm performs the worst when p_{n1} is around 0.1 and p_{n2} is around 0.08. This parameter setting leads to an imbalanced search behavior: the extremely low p_{n1} limits exploration in the early phase, undermining the algorithm's ability to discover diverse solutions, while the relatively high p_{n2} causes the algorithm to overemphasize local exploitation, possibly reinforcing suboptimal solutions. This combination significantly increases the risk of premature convergence and results in unsatisfactory scheduling performance.

Furthermore, the 3D surface plot shows that when both p_{n1} and p_{n2} are set to relatively low values, the performance of the algorithm also declines. This behavior is attributed to insufficient exploration and exploitation. A small p_{n1} limits the algorithm's ability to explore diverse regions in the early stages, increasing the risk of falling into local optima. Similarly, a small p_{n2} weakens the algorithm's ability to refine good solutions during the exploitation phase. When both parameters are low, the algorithm tends to lack the capability for both global and local search, resulting in suboptimal scheduling decisions and longer average job completion times.

These observations collectively highlight the importance of balancing exploration p_{n1} and exploitation p_{n2}. The algorithm achieves optimal performance only when p_{n1} is sufficiently large to ensure diverse exploration, while p_{n2} is carefully tuned to avoid over-exploitation. Both too small or imbalanced combinations would degrade the performance, underscoring the need for systematic parameter calibration in scheduling systems.

5.3.2 Sensitivity analysis of parameter *b*

In the following experiments, we set the value of p_{n1} to be 0.5 and the value of p_{n2} to be 0.04, respectively, based on the analysis conducted in Step 1. To investigate the sensitivity of parameter b in RTGS, we design a set of controlled experiments based on real-world workloads extracted from the Alibaba Cluster Trace. Specifically, three representative jobs are selected based on the number of subtasks: one with 400 subtasks representing a small job, one with 1,000 subtasks representing a medium-sized job, and one with 9,000 subtasks representing a large job. This classification allows us to evaluate the robustness of parameter b across different workload scales.

For each selected job, RTGS is executed under various settings of b, ranging from 0.2 to 3.0 in an increment of 0.2, resulting in 15 distinct values: $b \in \{0.2, 0.4, 0.6, \ldots, 2.8, 3.0\}$. For each value of b and for each job size, the algorithm is executed 30 times. After each run, we record the final job finish time (FFT) and the number of iterations required for RTGS to converge. Based on the collected data, we generate two box plots for visualization:

- The first plot depicts the distribution of FFT across different values of b for small, medium, and large jobs. In this plot, the x-axis represents the value of b, and the y-axis denotes the final finish time.

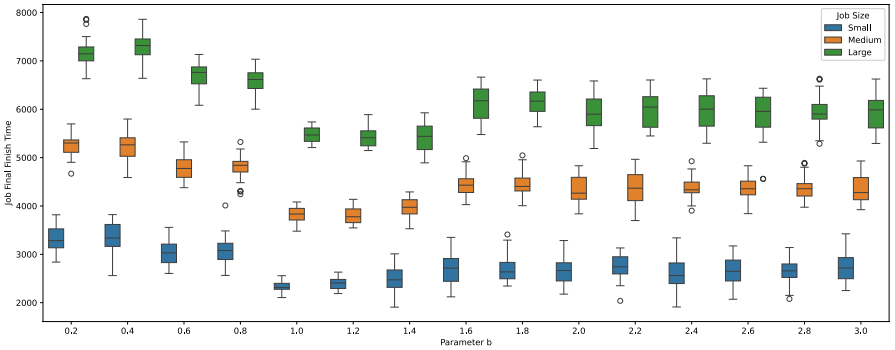


Fig. 7 Job final finish times under varying values of b for small, medium, and large jobs

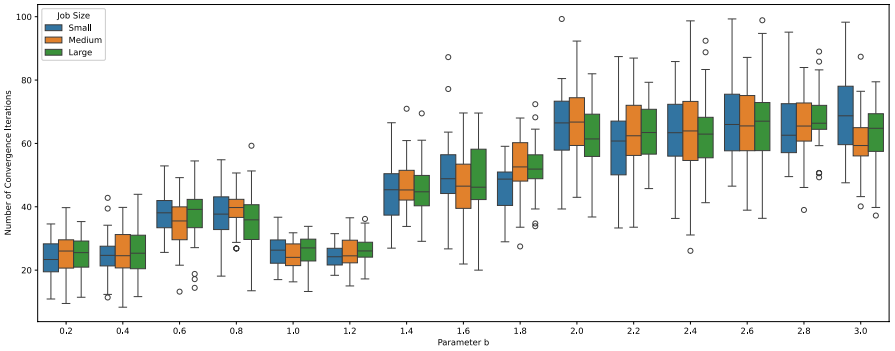


Fig. 8 Convergence iteration numbers under varying values of b for small, medium, and large jobs

- The second plot shows the corresponding convergence iteration numbers under the same values of b and job sizes, with x-axis for b and y-axis for the number of iterations until convergence.

To ensure the statistical validity of observed differences, we apply nonparametric statistical tests, the Friedman test [54] followed by the Nemenyi post hoc test [55], across the *FFT* results under different values of b. The Friedman test does not require the data to follow a normal distribution, making it particularly well suited for this type of scheduling experiments. This allows us to identify if variations in the performance are statistically significant.

Figure 7 presents the boxplots of *FFT* under varying values of b for three different job sizes. We observe that the value of b significantly affects the algorithm's performance. Specifically, when b is within the range of [1.0, 1.4], the algorithm tends to achieve better overall performance with relatively lower and more stable job completion times across all job sizes. In contrast, when b is too small (e.g., $b = 0.2$ or 0.4) or too large (e.g., $b \geq 1.6$), the completion times exhibit a higher variance and often increase substantially, indicating reduced scheduling efficiency and robustness.

Table 8 Significant difference analysis of final finish time and iterations across different values of b

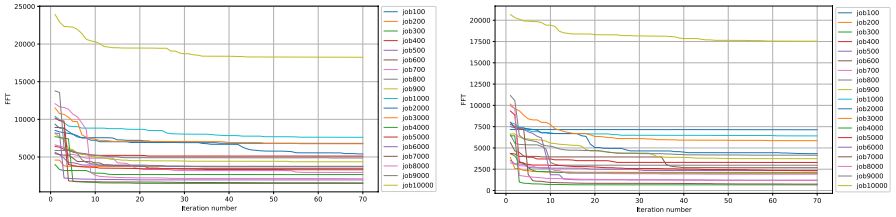
Job size	Metric	Friedman statistic (χ^2)	Friedman p value	Nemenyi significant pairs ($p < 0.05$)
Small	*FFT*	330.62	1.23e−59	$b = 0.2$ versus $b = 1.0$ (0.001), $b = 0.4$ versus $b = 1.2$ (0.001), $b = 1.4$ versus $b = 2.0$ (0.023)
	Iterations	283.45	2.45e−49	$b = 0.2$ versus $b = 2.0$ (0.001), $b = 0.6$ versus $b = 2.6$ (0.001)
Medium	*FFT*	298.17	1.89e−52	$b = 0.2$ versus $b = 1.0$ (0.001), $b = 0.4$ versus $b = 1.2$ (0.001), $b = 1.0$ versus $b = 2.0$ (0.045)
	Iterations	261.33	1.56e−44	$b = 0.2$ versus $b = 2.0$ (0.001), $b = 0.6$ versus $b = 2.4$ (0.001)
Large	*FFT*	275.89	3.78e−48	$b = 0.2$ versus $b = 1.0$ (0.001), $b = 0.4$ versus $b = 1.2$ (0.001), $b = 1.0$ versus $b = 2.0$ (0.032)
	Iterations	249.76	1.12e−42	$b = 0.2$ versus $b = 2.0$ (0.001), $b = 0.6$ versus $b = 2.6$ (0.001)

The convergence behavior of the algorithm is illustrated in Fig. 8, which shows the number of iterations required to reach convergence under each value setting of b. Similar to the makespan results, convergence tends to be faster and more stable when b is within the moderate range of [1.0, 1.4]. Extremely small or large values of b often result in either premature convergence or prolonged convergence, reflecting a poor balance between exploration and exploitation during the optimization process.

Based on the statistical results presented in Table 8, both the *FFT* and convergence iterations show statistically significant differences across different values of parameter b for all three job sizes: small, medium, and large, as confirmed by the extremely low p values from the Friedman tests. These findings suggest that a moderate value of b around 1.0 and 1.2 strikes a good balance between exploration and exploitation, yielding both efficient convergence and optimal scheduling performance.

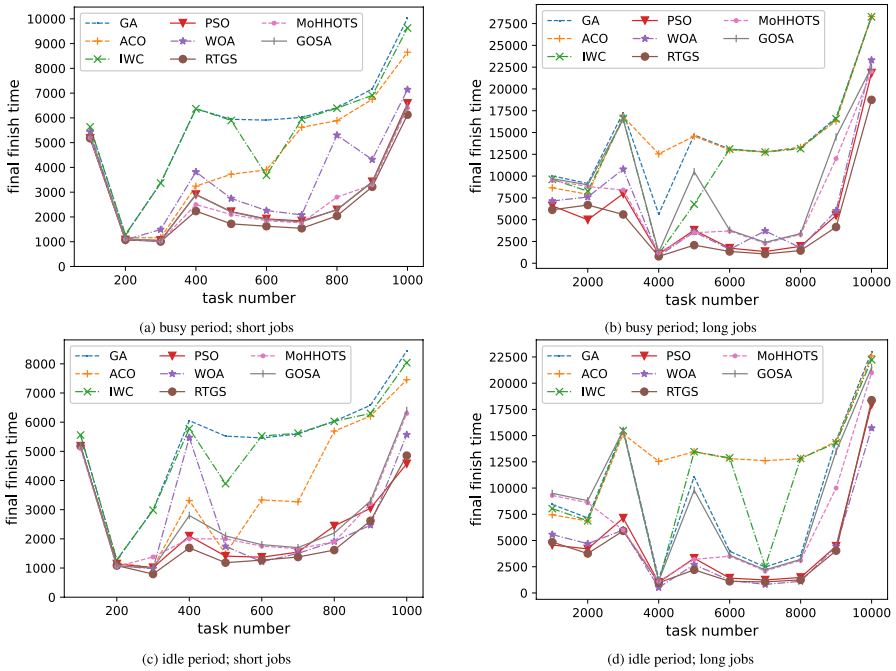
5.3.3 Sensitivity analysis of parameter *A*

Since k_{max} is the only manually configured variable in the computation of A, we conduct a sensitivity analysis to evaluate its effect on the algorithm's performance. The values of p_n and b are derived from the previous experiments. In the following experiments, we set the value of p_{n1} to be 0.5, p_{n2} to be 0.04 and b to be 1.0 to simplify the spiral update formula. To determine the number of maximum iterations, we conduct two sets of experiments. The first set examines the final completion times of all jobs selected from Alibaba Cluster Trace over each iteration during the busy period of the cluster. The second set examines the final completion times of all jobs selected from Alibaba Cluster Trace during the idle period of the cluster. The results are plotted in Fig. 9, where Fig. 9a shows the results during the busy period, and Fig. 9b shows the results during the idle period. We observe that the *FFT* value for



(a) Busy period (b) Idle period

Fig. 9 FFT across iterations during busy and idle periods



(a) busy period; short jobs (b) busy period; long jobs

(c) idle period; short jobs (d) idle period; long jobs

Fig. 10 Experiments based on workloads with DAG structures

all jobs stabilizes after approximately 10 iterations in Fig. 9a and 20 iterations in Fig. 9b. These results show that it takes longer to converge during the idle period. In the following experiments, we set the number of iterations to be 20.

5.4 Results of simulation experiments

Figures 10 and 11 plot the results from two sets of experiments with different job structures. Specifically, Fig. 10 shows the performance comparison for scheduling jobs with DAG structures, while Fig. 11 presents the results for scheduling jobs without DAG dependencies. This distinction allows us to evaluate the algorithm's effectiveness under both complex task dependencies and simpler,

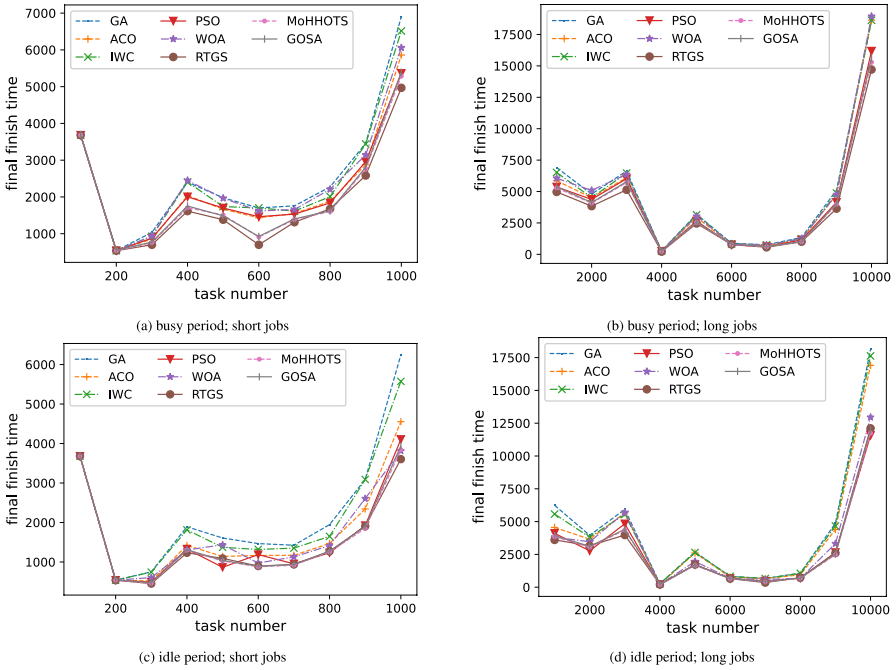


Fig. 11 Experiments based on workloads with independent tasks

linear job structures, providing a more comprehensive performance assessment. The horizontal axis represents the number of tasks, ranging from 100 to 1000 for short-running jobs and from 1000 to 10,000 for long-running jobs, all of which are extracted from the Alibaba dataset. The vertical axis represents the job final finish time. Note that a lower job final finish time indicates a better performance of the scheduler.

Compared with the original WOA algorithm, our experimental results show that WOA can sometimes achieve the same performance as RTGS, especially when scheduling a big job as shown in Fig. 10b, d. However, WOA is generally unstable. For instance, in Fig. 10c, when the task number is 400, the final finish time of WOA is 5474.0, whereas the final finish time of RTGS is only 1690.5. The primary reason for this significant instability is the operation between host identifiers, which is not directly related to the distance between two scheduling plans. This leads to a new scheduling plan being generated but not moving in the intended direction, and consequently, the WOA cannot effectively optimize scheduling plans. We transform the distance between scheduling plans into a different quantity as defined in Sect. 4.2, by leveraging the positive correlation between *FTG* and *SPD*. A new scheduling plan with a similar desired value of *SPD* can be created directly using a greedy algorithm. These experimental results clearly demonstrate the effectiveness of our transformation from *FTG* to *SPD* and the controlled iterative direction of the greedy algorithm. A weak direction control as in the original WOA algorithm would reduce the effectiveness of the WOA in the search for a suboptimal scheduling plan.

Table 9 Friedman test: algorithm performance on DAG tasks under different scenarios

Scenario	Friedman statistic (χ^2)	p value	Significant? ($\alpha = 0.05$)
Small tasks (busy)	54.30	< 0.001	Yes
Large tasks (busy)	54.99	< 0.001	Yes
Small tasks (idle)	53.58	< 0.001	Yes
Large tasks (idle)	53.64	< 0.001	Yes

Table 10 Nemenyi test: pairwise ranking differences for DAG task scheduling

	GA	ACO	IWC	PSO	WOA	RTGS	MoHHOTS	GOSA
Small tasks (busy)	7.70	5.30	7.30	2.30	4.10	1.90	2.60	2.70
Large tasks (busy)	7.40	7.30	6.60	2.40	3.60	1.90	4.20	4.60
Small tasks (idle)	7.70	5.10	7.20	2.50	2.70	1.60	3.30	3.80
Large tasks (idle)	7.50	7.40	6.90	2.10	2.50	1.90	4.60	4.50

Our approach achieves the best performance in both sets of experiments. The scheduling performance has a larger difference between these schedulers in Fig. 10 than in Fig. 11. This is because scheduling a job with a DAG structure is more challenging than scheduling a job without a specific structure. As shown in Fig. 10b, PSO has better scheduling ability than RTGS when the number of tasks is 2000. The reason is that this heuristic algorithm is based on a series of random numbers. PSO and RTGS have a better scheduling performance when scheduling jobs with DAG structures than the other four classical swarm intelligence schedulers as shown in Fig. 10. In the experiments with DAG structures, RTGS achieves up to 90% reduction of final finish time than GA, 93% reduction than ACO, 90% reduction than IWC, 45% reduction than PSO, 66% reduction than WOA, 46.7% reduction than GOSA, and 42.2% reduction than MoHHOTS. The superior performance of RTGS over MoHHOTS and GOSA is attributed to its dynamic prioritization mechanism and transformation-based search strategy. While MoHHOTS emphasizes multi-objective optimization using Harris Hawks behavior, its trade-off management relies heavily on stochastic Pareto strategies, which may not adapt quickly in dynamic cluster environments. In contrast, RTGS continuously refines the search direction via adaptive feedback on task completion states. Compared with GOSA, which is tailored for reconfigurable hardware and relies on simulated annealing to navigate graph-structured schedules, RTGS maintains a tighter control on convergence by using hope-value-guided prioritization and greedy corrections. These innovations enable RTGS to reduce the average finish time by more than 42% compared to MoHHOTS and 46.7% compared to GOSA when dealing with DAG-structured workloads.

When scheduling workloads without DAG structures as shown in Fig. 11, all of these algorithms in our experiments achieve a good performance because scheduling workloads without DAG structures is less challenging, but RTGS still achieves the best scheduling result, which reveals the strong search ability of RTGS. RTGS

Table 11 Friedman test: algorithm performance on non-DAG tasks under different scenarios

Scenario	Friedman statistic (χ^2)	p value	Significant? ($\alpha = 0.05$)
Small tasks (busy)	45.62	< 0.001	Yes
Large tasks (busy)	52.18	< 0.001	Yes
Small tasks (idle)	43.91	< 0.001	Yes
Large tasks (idle)	50.75	< 0.001	Yes

Table 12 Nemenyi test: pairwise ranking differences for non-DAG task scheduling

Scenario	RTGS	MoHHOTS	GOSA	PSO	ACO	WOA	IWC	GA
Small tasks (busy)	1.60	2.70	3.50	4.30	5.30	6.40	6.60	7.60
Large tasks (busy)	1.70	3.10	3.20	3.30	6.30	6.40	6.60	7.30
Small tasks (idle)	1.80	2.70	3.20	3.70	5.40	4.80	6.80	7.50
Large tasks (idle)	2.10	2.80	3.10	2.80	6.40	4.50	6.80	7.60

achieves up to 58% reduction of final finish time than GA, 50% reduction than ACO, 58% reduction than IWC, 50% reduction than PSO, 56% reduction than WOA, 10.05% reduction than GOSA, and 9.56% reduction than MoHHOTS in the experiments without DAG structures.

A comprehensive nonparametric analysis using the Friedman test and Nemenyi post hoc test on the eight scheduling algorithms across DAG and non-DAG tasks under busy and idle server conditions is presented in Tables 9, 10, 11, and 12. The Friedman test tables reveal significant performance differences ($p < 0.001$) in all scenarios. For DAG tasks, RTGS consistently achieves the lowest ranks of 1.60–1.90, significantly outperforming GA, IWC, and ACO, with PSO and WOA as strong contenders with ranks of 2.10–4.10. MoHHOTS and GOSA perform moderately, while GA and IWC have the highest ranks of 6.60–7.70, indicating their poor performance. For non-DAG tasks, RTGS again leads ranks of 1.60–2.10, followed closely by MoHHOTS, GOSA, and PSO with ranks of 2.70–4.30, all significantly better than GA, IWC, ACO, and WOA with ranks of 4.50–7.60. The consistent superiority of RTGS, robust performance of MoHHOTS, GOSA, and PSO, and consistent underperformance of GA and IWC across both task types and server conditions highlight RTGS as the optimal choice for scheduling, with MoHHOTS, GOSA, and PSO as viable alternatives.

5.5 Performance evaluation in realistic environments

We implement our scheduling algorithm and integrate it into Hadoop YARN system. The Hadoop software is deployed on a cluster consisting of 10 nodes, all of which operate on Ubuntu 20.04.6 LTS with Hadoop version 3.6.6 and Java version 1.8.0_191 installed. In this system, Apache Hadoop YARN serves as the critical resource management middleware, effectively managing and scheduling

Table 13 Server types in the Hadoop cluster

Server name	Type
master	RM
slave1, slave2, slave3, slave4, slave5, slave6, slave7, slave8, slave9	NM

Table 14 Hardware configurations of the Hadoop cluster

Server name	CPU	Memory (GB)
master, slave2, slave3, slave4, slave5, slave9	8 cores, 16 threads	32
slave1, slave6, slave7, slave8	6 cores, 12 threads	32

computational resources within the Hadoop cluster. Specifically, the resource manager (RM) acts as the central management unit, responsible for scheduling and allocating cluster resources to ensure that various jobs can utilize the resources fairly and efficiently. Each node in the cluster is monitored and managed by a node manager (NM), which tracks the local resource utilization in real time and reports this information back to the resource manager. The hardware configurations and system information for each node in the cluster are detailed in Tables 13 and 14.

In the experiments, we utilize the WordCount and TeraSort programs as benchmarks for Hadoop performance evaluation. The datasets for these programs are generated using the TeraGen tool,[6] designed for creating large datasets. Each Map task is configured to process 128 MB of data, with 1024 MB of memory allocated for both Map and Reduce tasks. To simulate a realistic user submission pattern, we prepare a set of random time intervals and submit 16 jobs according to these intervals. Eight of these jobs are WordCount, while the remaining eight are TeraSort, with data sizes ranging from 10 GB to 25 GB. Each job is scheduled 30 times, and the average *FFT* is calculated for each job across these 30 scheduling instances to mitigate random fluctuations.

The performance comparison of various algorithms in the Hadoop environment is presented in Fig. 12. Figure 12a shows the first 8 WordCount jobs along the horizontal axis. The vertical axis represents the average *FFT* of each job, calculated over 30 scheduling runs for each algorithm. Figure 12b plots the remaining 8 TeraSort jobs with the vertical axis showing the average *FFT* across 30 runs. RTGS consistently achieves the best performance for nearly all jobs as shown in Fig. 12, indicating its efficiency and strong adaptability to different workload patterns. MoHHOTS performs notably better than traditional heuristic algorithms such as GA, ACO, and IWC, which demonstrates the effectiveness of hybrid metaheuristics in improving scheduling performance. GOSA delivers decent performance in early jobs but begins to exhibit increased execution times in later more complex jobs.

[6] The TeraGen benchmarking tool is part of the standard Apache Hadoop distribution.

ACO, IWC, and GA show a clear sign of scaling inefficiency as task size and system complexity increase. Although WOA and PSO occasionally perform better than ACO or IWC, they show high variance in execution time, especially in scheduling the TeraSort jobs.

To evaluate the overall performance of various scheduling algorithms across multiple job scenarios, we employ the Friedman nonparametric statistical test for performance comparison. Tables 16 and 17 present the Friedman test and Nemenyi post hoc test results, including the average rank of each algorithm, which is calculated by sorting the execution times across all jobs for each algorithm. A lower rank indicates a better performance. The Friedman test results show that there is a significant statistical difference in execution times among the algorithms ($p < 0.05$). This indicates that the performance differences between these algorithms are not merely due to random fluctuations but represent true statistical differences. According to the Nemenyi post hoc test, RTGS significantly outperforms all other algorithms. Among the rest, MoHHOTS ranks the second, showing a competitive performance but still with statistically inferior results compared with RTGS.

Table 15 presents a detailed statistical analysis of the eight scheduling algorithms evaluated across all three distinct environments: a real Hadoop cluster and simulated environments with and without DAG task structures. The analysis is based on the Friedman test followed by the Nemenyi post hoc test, with average ranks and significance comparisons against the baseline algorithm, RTGS, under different critical difference (CD) thresholds: $CD = 2.63$ for Hadoop, $CD = 3.32$ for simulations. The simulated environments include four scenarios: small tasks busy environment (SB), large tasks busy environment (LB), small tasks idle environment (SI), and large tasks idle environment (LI). Below is a comprehensive analysis of the results, highlighting performance trends, algorithm strengths, and implications for scheduling applications. RTGS consistently emerges as the top-performing algorithm, with the lowest average ranks across all scenarios: 1.44 in Hadoop, 1.60–1.90 in DAG simulations, and 1.60–2.10 in non-DAG simulations. This suggests that RTGS is highly efficient in minimizing job execution time, regardless of task structures (DAG or non-DAG) and server conditions (busy or idle). Conversely, GA and IWC are consistently the least effective, with high ranks of 6.25–7.38 in Hadoop, 6.60–7.70 in DAG, and 6.60–7.60 in non-DAG, indicating significantly worse performance compared with RTGS. The remaining algorithms: MoHHOTS, GOSA, PSO, WOA, and ACO, form a spectrum of mid-tier performers, with varying strengths depending on the environment and scenario. RTGS and PSO consistently perform well across all scenarios. MoHHOTS and GOSA are reliable mid-tier performers but struggle with large tasks under busy conditions. ACO, IWC, and GA consistently underperform, with GA being the least effective, likely due to poor handling of DAG dependencies.

5.6 Practical significance in real-world DAG workloads

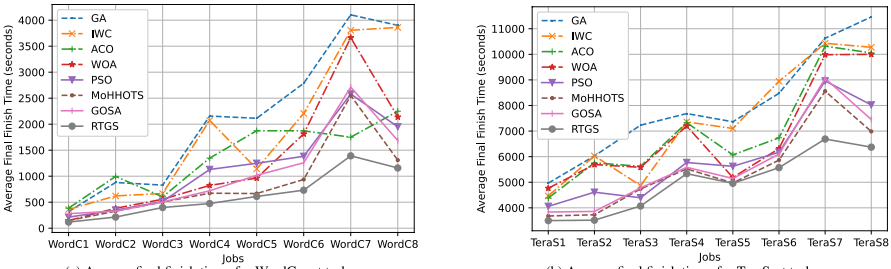
DAG-structured workloads are prevalent in numerous modern real-world applications, particularly in large-scale data processing and machine learning fields. For

Table 15 Average ranks and significance analysis of scheduling algorithms in different scenarios (baseline: RTGS)

Algorithm	Hadoop (real cluster)		Simulation with DAG					Simulation without DAG				
	Avg. rank	CD=2.63	Avg. rank in subtasks (CD = 3.32)				Signif	Avg. rank in subtasks (CD=3.32)				Signif.
			SB	LB	SI	LI	versus RTGS	SB	LB	SI	LI	versus RTGS
RTGS	1.44	–	1.90	1.90	1.60	1.90	–	1.60	1.70	1.80	2.10	-
MoHHOTS	2.56	No	2.60	4.20	3.30	4.60	No	2.70	3.10	2.70	2.80	No
GOSA	3.81	No	2.70	4.60	3.80	4.50	No	3.50	3.20	3.20	3.10	No
PSO	4.00	No	2.30	2.40	2.50	2.10	No	4.30	3.30	3.70	2.80	No
WOA	4.81	Yes	4.10	3.60	2.70	2.50	No	6.40	6.40	4.80	4.50	Yes (except LI)
ACO	5.75	Yes	5.30	7.30	5.10	7.40	Yes (except LI)	5.30	6.30	5.40	6.40	Yes (except SB)
IWC	6.25	Yes	7.30	6.60	7.20	6.90	Yes	6.60	6.60	6.80	6.80	Yes
GA	7.38	Yes	7.70	7.40	7.70	7.50	Yes	7.60	7.30	7.50	7.60	Yes

"Signif. versus RTGS" indicates if the algorithm is significantly worse than RTGS under the corresponding CD threshold

SB, small tasks (busy); LB, large tasks (busy); SI, small tasks (idle); LI, large tasks (idle)



(a) Average final finish times for WordCount tasks

(b) Average final finish times for TeraSort tasks

Fig. 12 Performance comparison of scheduling algorithms on Hadoop using WordCount and TeraSort benchmarks

instance, deep learning training pipelines, data preprocessing frameworks (e.g., Apache Spark, TensorFlow), and large model fine-tuning workflows inherently adopt DAG representations to capture task dependencies. In such systems, task execution order and resource allocation are critical to overall efficiency, especially under resource-constrained or heterogeneous computing environments. The proposed RTGS algorithm demonstrates strong adaptability and effectiveness in these contexts by significantly reducing the final finish time through its structure-aware scheduling strategy.

Furthermore, the experimental results in Sect. 5 validate the algorithm's scalability and robustness across varying workload sizes and system states. Specifically, RTGS achieves up to 90% improvement in final finish time over traditional metaheuristic algorithms in DAG-structured scenarios. These substantial gains underline its potential to support real-time or near-real-time processing in production-scale cloud systems.

In addition to experimental performance, RTGS is designed with scalability. It operates within the distributed architecture of Hadoop YARN, where scheduling is managed hierarchically by a central ResourceManager and multiple NodeManagers. This naturally supports extension to larger clusters. Moreover, RTGS relies on lightweight hybrid metaheuristics and parallelizable evaluation strategies that maintain low computational overhead, ensuring fast response times even as node count and workload size scale up. These design features enable RTGS to maintain scheduling efficiency and decision quality in larger and more complex environments, such as clusters with hundreds or thousands of nodes. We also plan

Table 16 Friedman test: algorithm performance in the Hadoop system

Friedman statistic (χ^2)	p value	Significant? ($\alpha = 0.05$)
88.19	< 0.001	Yes

Table 17 Nemenyi test: pairwise ranking differences in the Hadoop system

	RTGS	MoHHOTS	GOSA	PSO	WOA	ACO	IWC	GA
Average rank	1.44	2.56	3.81	4.00	4.81	5.75	6.25	7.38

to further explore RTGS in elastic cloud-native environments, including Kubernetes-based container orchestration systems, to validate its practical scalability under diverse deployment conditions.

In contrast to existing approaches such as GOSA and MoHHOTS, which focus on reconfiguration-aware scheduling and energy-delay optimization, respectively, RTGS provides a more generalized and scalable scheduling mechanism that directly addresses the core challenge of dependency-preserving execution while maintaining high efficiency across heterogeneous workloads. As DAG structures become increasingly dominant in AI and big data systems, the proposed method offers a highly practical and adaptable scheduling solution in emerging real-world computing environments.

6 Conclusion and future work

In this work, we investigated a task scheduling problem, LRW-TS, for DAG-structured long-running workloads on a parallel computing platform with virtual or physical servers. This problem is formulated as an NIP problem and proved to be NP-complete. We revealed an important positive correlation between *SPD* and *FTG*, and by leveraging this finding, we proposed a novel algorithm, RTGS, based on the WOA algorithm to solve LRW-TS. In RTGS, we derived a function to generate the humpback's next step, and designed a greedy algorithm to satisfy each server's load demand. Extensive experimental results based on real-life workload traces show that RTGS achieves a better performance in comparison with seven state-of-the-art baseline algorithms.

We plan to improve the greedy algorithm and modify the *SPD* function to generate new scheduling schemes that can precisely control the direction of each iteration. Another future interest about this work is to develop a systematic rule of judgment to decide an appropriate set of workloads RTGS is best suited for and provide a better quality of service for task scheduling in big data computing systems.

Author contributions Nana Du was involved in designed the research methodology, collected and analyzed the data, and contributed to the writing of the manuscript. Yudong Ji helped in conducted literature review, collected and analyzed the data, contributed to the experimental design. Chase Wu provided expertise in the subject matter, guided the formulation of the problem, advised the design of the solution, and edited the manuscript. Aiqin Hou provided expertise in the subject matter, supervised the research project, and reviewed the final draft. Weike Nie provided expertise in the subject matter and reviewed the final draft.

Data availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Garefalakis P, Karanasos K, Pietzuch P, Suresh A, Rao S (2018) Medea: scheduling of long running applications in shared production clusters. In: Proceedings of the Thirteenth EuroSys Conference, pp 1–13
2. Vavilapalli VK, Murthy AC, Douglas C, Agarwal S, Konar M, Evans R, Graves T, Lowe J, Shah H, Seth S et al (2013) Apache hadoop yarn: yet another resource negotiator. In: Proceedings of the 4th Annual Symposium on Cloud Computing, pp 1–16
3. Google CNCF (2017) What is kubernetes, Website. https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/
4. Hindman B, Konwinski A, Zaharia M, Ghodsi A, Joseph AD, Katz R, Shenker S, Stoica I (2011) Mesos: a platform for fine-grained resource sharing in the data center. In: 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)
5. Sun P, Guo Z, Wang J, Li J, Lan J, Hu Y (2021) Deepweave: accelerating job completion time with deep reinforcement learning-based coflow scheduling. In: Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence (IJCAI 29), pp 3314–3320
6. Mao H, Schwarzkopf M, Venkatakrishnan SB, Meng Z, Alizadeh M (2019) Learning scheduling algorithms for data processing clusters. In: Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM 2019), pp 270–288
7. Mirjalili S, Lewis A (2016) The whale optimization algorithm. Adv Eng Softw 95:51–67
8. Shivahare BD, Singh M, Gupta A, Ranjan S, Pareta D, Sahu BM (2021) Survey paper: whale optimization algorithm and its variant applications. In: 2021 International Conference on Innovative Practices in Technology and Management (ICIPTM), IEEE, pp 77–82
9. Mohammed HM, Umar SU, Rashid TA (2019) A systematic and meta-analysis survey of whale optimization algorithm. Comput Intell Neurosci 2019(1):8718571
10. Banerjee S, Jha S, Kalbarczyk Z, Iyer R (2020) Inductive-bias-driven reinforcement learning for efficient schedules in heterogeneous clusters. In: International Conference on Machine Learning, pp 629–641
11. Mondal SS, Sheoran N, Mitra S (2021) Scheduling of time-varying workloads using reinforcement learning. Proc AAAI Conf Artif Intell 35:9000–9008
12. Marcus R, Papaemmanouil O (2016) Wisedb: a learning-based workload management advisor for cloud databases, arXiv preprint arXiv:1601.08221
13. Chen Y, Brock B, Porumbescu S, Buluç A, Yelick K, Owens JD (2022) Scalable irregular parallelism with GPUs: getting CPUs out of the way. In: SC22: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, pp 1–16
14. Cheng D, Zhou X, Lama P, Wu J, Jiang C (2017) Cross-platform resource scheduling for spark and MapReduce on yarn. IEEE Trans Comput 66(8):1341–1353
15. Yoo D, Sim KM (2011) A comparative review of job scheduling for MapReduce. In: 2011 IEEE International Conference on Cloud Computing and Intelligence Systems. IEEE, pp 353–358
16. Patil A, Bagban T, Pande A (2015) Recent job scheduling algorithms in hadoop cluster environments: a survey. Int J Adv Res Comput Commun Eng 4(2)
17. Arpaci-Dusseau RH, Arpaci-Dusseau AC (2018) Operating systems: three easy pieces. Arpaci-Dusseau Books, LLC
18. Alibaba web services. https://www.alibaba.com/
19. Google cloud platform. https://cloud.google.com/

20. Du J, Wei J, Jiang J, Cheng S, Huang D, Chen Z, Lu Y (2024) Liger: interleaving intra-and inter-operator parallelism for distributed large model inference. In: Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pp 42–54
21. Wang C, Sun D, Bai Y (2023) Pipad: pipelined and parallel dynamic GNN training on GPUs. In: Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pp 405–418
22. Osama M, Porumbescu SD, Owens JD (2023) A programming model for GPU load balancing. In: Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pp 79–91
23. Stec R, Novak A, Sucha P, Hanzalek Z (2019) Scheduling jobs with stochastic processing time on parallel identical machines. In: IJCAI, pp 5628–5634
24. Angelopoulos S, Jin S (2019) Earliest-completion scheduling of contract algorithms with end guarantees. In: IJCAI, pp 5493–5499
25. Dong H, Wang B, Qiao B, Xing W, Luo C, Qin S, Lin Q, Zhang D, Virdi G, Moscibroda T (2021) Predictive job scheduling under uncertain constraints in cloud computing. In: IJCAI, pp 3627–3634
26. Ali A, Shah SAA, Al Shloul T, Assam M, Ghadi YY, Lim S, Zia A (2024) Multiobjective Harris Hawks optimization-based task scheduling in cloud-fog computing. IEEE Internet Things J 11(13):24334–24352
27. Mollajafari M (2023) An efficient lightweight algorithm for scheduling tasks onto dynamically reconfigurable hardware using graph-oriented simulated annealing. Neural Comput Appl 35(24):18035–18057
28. Schardl TB, Lee I-TA (2023) Opencilk: a modular and extensible software infrastructure for fast task-parallel code. In: Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pp 189–203
29. Godet A, Lorca X, Hebrard E, Simonin G (2020) Using approximation within constraint programming to solve the parallel machine scheduling problem with additional unit resources. Proc AAAI Conf Artif Intell 34:1512–1519
30. Joe-Wong C, Sen S, Lan T, Chiang M (2013) Multiresource allocation: fairness-efficiency tradeoffs in a unifying framework. IEEE/ACM Trans Netw 21(6):1785–1798
31. Grandl R, Ananthanarayanan G, Kandula S, Rao S, Akella A (2014) Multi-resource packing for cluster schedulers. ACM SIGCOMM Comput Commun Rev 44(4):455–466
32. Delimitrou C, Kozyrakis C (2014) Quasar: resource-efficient and qos-aware cluster management. ACM SIGPLAN Notices 49(4):127–144
33. Delimitrou C (2013) Paragon: Qos-aware scheduling for heterogeneous datacenters. ACM SIGPLAN Notices 48(4):77–88
34. Ghafari R, Mansouri N (2024) Improved Harris Hawks optimizer with chaotic maps and opposition-based learning for task scheduling in cloud environment. Clust Comput 27(2):1421–1469
35. Boutin E, Ekanayake J, Lin W, Shi B, Zhou J, Qian Z, Wu M, Zhou L (2014) Apollo: scalable and coordinated scheduling for cloud-scale computing. In: 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14), pp 285–300
36. Gog I, Schwarzkopf M, Gleave A, Watson RN, Hand S (2016) Firmament: fast, centralized cluster scheduling at scale. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp 99–115
37. Fu Y, Liu L, Wang H, Cheng Y, Chen S (2022) SFS: smart OS scheduling for serverless functions. In: SC22: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, pp 1–16
38. Boveiri HR, Khayami R, Elhoseny M, Gunasekaran M (2019) An efficient swarm-intelligence approach for task scheduling in cloud-based internet of things applications. J Amb Intell Hum Comput 10(9):3469–3479
39. Boveiri HR, Javidan R, Khayami R (2021) An intelligent hybrid approach for task scheduling in cluster computing environments as an infrastructure for biomedical applications. Expert Syst 38(1):e12536
40. Boveiri HR (2020) An enhanced cuckoo optimization algorithm for task graph scheduling in cluster-computing systems. Soft Comput 24(13):10075–10093
41. Ibm ilog cplex optimization. https://www.ibm.com/docs/en/icos
42. Karp RM (1972) Reducibility among combinatorial problems. In: Complexity of Computer Computations. Springer, pp 85–103

43. Cormen TH, Leiserson CE, Rivest RL, Stein C (2001) Topological sort. In: Introduction to Algorithms, 2nd ed. MIT Press, Ch. 22.4, pp 549–552
44. Cormen TH, Leiserson CE, Rivest RL, Stein C (2001) Single-source shortest paths in directed acyclic graphs. In: Introduction to Algorithms, 2nd ed. MIT Press, Ch. 24.2, pp 592–595
45. Galton F (1886) Regression towards mediocrity in hereditary stature. J Anthropol Inst G B Irel 15:246–263
46. Overholser BR, Sowinski KM (2008) Biostatistics primer: part 2. Nutr Clin Pract 23(1):76–84
47. Goldbogen JA, Friedlaender AS, Calambokidis J, McKenna MF, Simon M, Nowacek DP (2013) Integrative approaches to the study of baleen whale diving behavior, feeding performance, and foraging ecology. BioScience 63(2):90–100
48. Watkins WA, Schevill WE (1979) Aerial observation of feeding behavior in four baleen whales: *Eubalaena glacialis*, *Balaenoptera borealis*, *Megaptera novaeangliae*, and *Balaenoptera physalus*. J Mammal 60(1):155–163
49. Chen X, Cheng L, Liu C, Liu Q, Liu J, Mao Y, Murphy J (2020) A WOA-based optimization approach for task scheduling in cloud computing systems. IEEE Syst J 14(3):3117–3128
50. Dorigo M, Di Caro G (1999) Ant colony optimization: a new meta-heuristic. In: Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406), vol 2. IEEE, pp 1470–1477
51. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: Proceedings of ICNN'95-International Conference on Neural Networks, vol 4. IEEE, pp 1942–1948
52. Gen M, Cheng R (1999) Genetic algorithms and engineering optimization, vol 7. Wiley
53. Fritsch FN, Carlson RE (1980) Monotone piecewise cubic interpolation. SIAM J Numer Anal 17(2):238–246
54. Friedman M (1937) The use of ranks to avoid the assumption of normality implicit in the analysis of variance. J Am Stat Assoc 32(200):675–701
55. Nemenyi PB (1963) Distribution-free multiple comparisons. Princeton University

Authors and Affiliations

Nana Du[1] · Yudong Ji[1] · Chase Wu[2] · Aiqin Hou[1] · Weike Nie[1]

✉ Chase Wu
chase.wu@njit.edu

✉ Aiqin Hou
houaiqin@nwu.edu.cn

Nana Du
dunana@stumail.nwu.edu.cn

Yudong Ji
jiyudong@stumail.nwu.edu.cn

Weike Nie
weikenie@nwu.edu.cn

[1] School of Information Science and Technology, Northwest University, Xi'an 710100, Shaanxi, China

[2] Department of Data Science, New Jersey Institute of Technology, Newark, NJ 07102, USA